SAW User Manual

The SAW Development Team

Apr 07, 2026

Contents

1 Overview **1**
- 1.1 SAW Use Cases . . . 2
- 1.2 SAW Terminology . . . 2
- 1.3 Running Example . . . 2

2 Structure of SAWScript **3**
- 2.1 Parts of a SAW Script . . . 4
- 2.2 A First Simple Example . . . 4
- 2.3 Syntax . . . 5
- 2.4 Basic Types and Values . . . 5
- 2.5 Basic Expression Forms . . . 6
- 2.6 Other Basic Functions . . . 9
- 2.7 REPL Actions . . . 10
- 2.8 Further built-in functions and commands . . . 11
- 2.9 Experimental and deprecated functions and commands . . . 11

3 Invoking SAW **13**

4 Cryptol and its Role in SAW **15**

5 Loading Code **21**
- 5.1 Loading LLVM . . . 21
- 5.2 Loading Java . . . 21
- 5.3 Loading MIR . . . 22
- 5.4 Notes on Compiling Code for SAW . . . 22
- 5.5 Direct Extraction . . . 25
- 5.6 Notes on C++ Analysis . . . 26

6 Analyzing Hardware Circuits using Yosys **27**
- 6.1 Processing VHDL With Yosys . . . 27

6.2 Example: Ripple-Carry Adder . . . 28
6.3 API Reference . . . 32

7 Creating Symbolic Variables **35**

8 Symbolic Execution **39**

9 Symbolic Termination **41**

10 The Term Type **43**

11 Specification-Based Verification **45**
11.1 Running a Verification . . . 45
11.2 Structure of a Specification . . . 47
11.3 Creating Fresh Variables . . . 48
11.4 The LLVMValue, JVMValue, and MIRValue Types . . . 50
11.5 Executing . . . 53
11.6 Return Values . . . 53
11.7 A First Simple Example (Revisited) . . . 54
11.8 Compositional Verification . . . 54
11.9 Specifying Heap Layout . . . 60
11.10 Specifying Heap Values . . . 62
11.11 Working with Compound Types . . . 64
11.12 Global variables . . . 76
11.13 Preconditions and Postconditions . . . 79
11.14 Assuming specifications . . . 80
11.15 A Heap-Based Example . . . 80
11.16 Using Ghost State . . . 82
11.17 An Extended Example . . . 83
11.18 Verifying Cryptol FFI functions . . . 88
11.19 Multi-Step Proofs and Loop Invariants with Cuts . . . 90

12 Bisimulation Prover **95**
12.1 Bisimulation Example . . . 96
12.2 Understanding the proof goals . . . 99
12.3 Limitations . . . 100

13 Transforming Term Values **103**
13.1 Rewriting . . . 103
13.2 Folding and Unfolding . . . 106
13.3 Other Built-in Transformation and Inspection Functions . . . 107
13.4 Loading and Storing Terms . . . 108

14 Proofs about Terms **109**
14.1 Automated Tactics . . . 109
14.2 Proof Script Diagnostics . . . 110

14.3 Rewriting in Proof Scripts . . . 111
14.4 Other Transformations . . . 111
14.5 Caching Solver Results . . . 112
14.6 Other External Provers . . . 114
14.7 Offline Provers . . . 115
14.8 Finishing Proofs without External Solvers . . . 115
14.9 Multiple Goals . . . 116
14.10 Proof Failure and Satisfying Assignments . . . 116
14.11 AIG Values and Proofs . . . 117

15 Extraction to the Rocq theorem prover **119**
15.1 Support Library . . . 119
15.2 Cryptol module extraction . . . 120
15.3 Proofs involving uninterpreted functions . . . 121
15.4 Translation limitations and caveats . . . 121

16 Formal Deprecation Process **123**

17 Appendices **125**
17.1 Glossary . . . 125
17.2 Command Reference . . . 125
17.3 REPL Reference . . . 125
17.4 Deprecated Items . . . 128
17.5 SAWScript Language Reference . . . 128

CHAPTER 1

Overview

The Software Analysis Workbench (SAW) is a tool for constructing mathematical models of the computational behavior of software, transforming these models, and proving properties about them.

SAW can currently construct models of a subset of programs written in Cryptol, LLVM (and therefore C), JVM byte code (and therefore Java), and MIR (and therefore Rust). The models take the form of typed functional programs, so in a sense SAW can be considered a translator from imperative programs to their functional equivalents. Various external proof tools, including a variety of SAT and SMT solvers, can be used to prove properties about the functional models. SAW can construct models from arbitrary Cryptol programs, and from C and Java programs that have fixed-size inputs and outputs and that terminate after a fixed number of iterations of any loop (or a fixed number of recursive calls). One common use case is to verify that an algorithm specification in Cryptol is equivalent to an algorithm implementation in C or Java.

The process of extracting models from programs, manipulating them, forming queries about them, and sending them to external provers is orchestrated using a special purpose language called SAWScript. SAWScript is a typed functional language with support for sequencing of imperative commands.

The rest of this document first describes how to use the SAW tool, `saw`, and outlines the structure of the SAWScript language and its relationship to Cryptol. It then presents the SAWScript commands that transform functional models and prove properties about them. Finally, it describes the specific commands available for constructing models from imperative programs.

1.1 SAW Use Cases

Warning

This section is under construction!

1.2 SAW Terminology

Warning

This section is under construction!

1.3 Running Example

Warning

This section is under construction!

CHAPTER 2

Structure of SAWScript

A SAWScript program consists, at the top level, of a sequence of commands to be executed in order. Each command is terminated with a semicolon. For example, the `print` command displays a textual representation of its argument. Suppose the following text is stored in the file `print.saw`:

```
print 3;
```

The command `saw print.saw` will then yield output similar to the following:

```
Loading module Cryptol
Loading file "print.saw"
3
```

The same code can be run from the interactive REPL:

```
sawscript> print 3;
3
```

At the REPL, terminating semicolons can be omitted:

```
sawscript> print 3
3
```

To make common use cases simpler, bare values at the REPL are treated as if they were arguments to `print`:

```
sawscript> 3
3
```

One SAWScript file can be included in another using the `include` command, which takes the name of the file to be included as an argument. For example:

```
sawscript> include "print.saw"
Loading file "print.saw"
3
```

Typically, included files are used to import definitions, not perform side effects like printing. However, as you can see, if any commands with side effects occur at the top level of the imported file, those side effects will occur during import.

2.1 Parts of a SAW Script

> **Warning**
>
> This section is under construction!

2.2 A First Simple Example

> **Warning**
>
> This section is under construction!

To get started with SAW, let's see what it takes to verify simple programs that do not use pointers (or that use them only internally). Consider, for instance the C program that adds its two arguments together:

```
#include <stdint.h>
uint32_t add(uint32_t x, uint32_t y) {
    return x + y;
}
```

We can specify this function's expected behavior as follows:

```
let add_setup = do {
    x <- llvm_fresh_var "x" (llvm_int 32);
    y <- llvm_fresh_var "y" (llvm_int 32);
    llvm_execute_func [llvm_term x, llvm_term y];
```

(continues on next page)

(continued from previous page)

```
    llvm_return (llvm_term {{ x + y : [32] }});
};
```

We can then compile the C file `add.c` into the bitcode file `add.bc` and verify it with ABC:

```
m <- llvm_load_module "add.bc";
add_ms <- llvm_verify m "add" [] false add_setup abc;
```

2.3 Syntax

The syntax of SAWScript is reminiscent of functional languages such as Cryptol, Haskell and ML. In particular, functions are applied by writing them next to their arguments rather than by using parentheses and commas. Rather than writing `f(x, y)`, write `f x y`.

Comments are written as in C, Java, and Rust (among many other languages). All text from `//` until the end of a line is ignored. Additionally, all text between `/*` and `*/` is ignored, regardless of whether the line ends.

2.4 Basic Types and Values

All values in SAWScript have types, and these types are determined and checked before a program runs (that is, SAWScript is statically typed). The basic types available are similar to those in many other languages.

- The `Int` type represents unbounded mathematical integers. Integer constants can be written in decimal notation (e.g., `42`), hexadecimal notation (`0x2a`), and binary (`0b00101010`). However, unlike many languages, integers in SAWScript are used primarily as constants. Arithmetic is usually encoded in Cryptol, as discussed in the next section.
- The Boolean type, `Bool`, contains the values `true` and `false`, like in many other languages. As with integers, computations on Boolean values usually occur in Cryptol.
- Values of any type can be aggregated into tuples. For example, the value `(true, 10)` has the type `(Bool, Int)`.
- Values of any type can also be aggregated into records, which are exactly like tuples except that their components have names. For example, the value `{ b = true, n = 10 }` has the type `{ b : Bool, n : Int }`.
- A sequence of values of the same type can be stored in a list. For example, the value `[true, false, true]` has the type `[Bool]`.
- Strings of textual characters can be represented in the `String` type. For example, the value `"example"` has type `String`.

- The "unit" type, written `()`, is essentially a placeholder, similar to `void` in languages like C and Java. It has only one value, also written `()`. Values of type `()` convey no information. We will show in later sections several cases where this is useful.
- Functions are given types that indicate what type they consume and what type they produce. For example, the type `Int -> Bool` indicates a function that takes an `Int` as input and produces a `Bool` as output. Functions with multiple arguments use multiple arrows. For example, the type `Int -> String -> Bool` indicates a function in which the first argument is an `Int`, the second is a `String`, and the result is a `Bool`. It is possible, but not necessary, to group arguments in tuples, as well, so the type `(Int, String) -> Bool` describes a function that takes one argument, a pair of an `Int` and a `String`, and returns a `Bool`.

SAWScript also includes some more specialized types that do not have straightforward counterparts in most other languages. These will appear in later sections.

2.5 Basic Expression Forms

One of the key forms of top-level command in SAWScript is a *binding*, introduced with the `let` keyword, which gives a name to a value. For example:

```
sawscript> let x = 5
sawscript> x
5
```

Bindings can have parameters, in which case they define functions. For instance, the following function takes one parameter and constructs a list containing that parameter as its single element.

```
sawscript> let f x = [x]
sawscript> f "text"
["text"]
```

Functions themselves are values and have types. The type of a function that takes an argument of type `a` and returns a result of type `b` is `a -> b`.

Function types are typically inferred, as in the example `f` above. In this case, because `f` only creates a list with the given argument, and because it is possible to create a list of any element type, `f` can be applied to an argument of any type. We say, therefore, that `f` is *polymorphic*. Concretely, we write the type of `f` as `{a} a -> [a]`, meaning it takes a value of any type (denoted `a`) and returns a list containing elements of that same type. This means we can also apply `f` to `10`:

```
sawscript> f 10
[10]
```

However, we may want to specify that a function has a more specific type. In this case, we could restrict `f` to operate only on `Int` parameters.

```
sawscript> let f (x : Int) = [x]
```

This will work identically to the original `f` on an `Int` parameter:

```
sawscript> f 10
[10]
```

However, it will fail for a `String` parameter:

```
sawscript> f "text"

type mismatch: String -> t.0 and Int -> [Int]
 at "_" (REPL)
mismatched type constructors: String and Int
```

Type annotations can be applied to any expression. The notation `(e : t)` indicates that expression `e` is expected to have type `t` and that it is an error for `e` to have a different type. Most types in SAWScript are inferred automatically, but specifying them explicitly can sometimes enhance readability.

Because functions are values, functions can return other functions. We make use of this feature when writing functions of multiple arguments. Consider the function `g`, similar to `f` but with two arguments:

```
sawscript> let g x y = [x, y]
```

Like `f`, `g` is polymorphic. Its type is `{a} a -> a -> [a]`. This means it takes an argument of type `a` and returns a *function* that takes an argument of the same type `a` and returns a list of `a` values. We can therefore apply `g` to any two arguments of the same type:

```
sawscript> g 2 3
[2,3]
sawscript> g true false
[true,false]
```

But type checking will fail if we apply it to two values of different types:

```
sawscript> g 2 false

type mismatch: Bool -> t.0 and Int -> [Int]
 at "_" (REPL)
mismatched type constructors: Bool and Int
```

So far we have used two related terms, *function* and *command*, and we take these to mean slightly different things. A function is any value with a function type (e.g., `Int -> [Int]`). A command is any value with a special command type (e.g. `TopLevel ()`, as shown below). These special types allow us to restrict command usage to specific contexts, and are also *parameterized* (like the list type). Most but not all commands are also functions.

The most important command type is the `TopLevel` type, indicating a command that can run at the top level (directly at the REPL, or as one of the top level commands in a script file). The `print` command has the type `{a} a -> TopLevel ()`, where `TopLevel ()` means that it is a command that runs in the `TopLevel` context and returns a value of type `()` (that is, no useful information). In other words, it has a side effect (printing some text to the screen) but doesn't produce any information to use in the rest of the SAWScript program. This is the primary usage of the `()` type.

It can sometimes be useful to bind a sequence of commands together. This can be accomplished with the `do { ... }` construct. For example:

```
sawscript> let print_two = do { print "first"; print "second"; }
sawscript> print_two
first
second
```

The bound value, `print_two`, has type `TopLevel ()`, since that is the type of its last command.

Note that in the previous example the printing doesn't occur until `print_two` directly appears at the REPL. The `let` expression does not cause those commands to run. The construct that *runs* a command is written using the `<-` operator. This operator works like `let` except that it says to run the command listed on the right hand side and bind the result, rather than binding the variable to the command itself. Using `<-` instead of `let` in the previous example yields:

```
sawscript> print_two <- do { print "first"; print "second"; }
first
second
sawscript> print print_two
()
```

Here, the `print` commands run first, and then `print_two` gets the value returned by the second `print` command, namely `()`. Any command run without using `<-` at either the top level of a script or within a `do` block discards its result. However, the REPL prints the result of any command run without using the `<-` operator.

In some cases it can be useful to have more control over the value returned by a `do` block. The `return` command allows us to do this. For example, say we wanted to write a function that would print a message before and after running some arbitrary command and then return the result of that command. We could write:

```
let run_with_message msg c =
  do {
    print "Starting.";
    print msg;
    res <- c;
    print "Done.";
    return res;
```

(continues on next page)

(continued from previous page)

```
  };

x <- run_with_message "Hello" (return 3);
print x;
```

If we put this script in `run.saw` and run it with `saw`, we get something like:

```
Loading module Cryptol
Loading file "run.saw"
Starting.
Hello
Done.
3
```

Note that it ran the first `print` command, then the caller-specified command, then the second `print` command. The result stored in `x` at the end is the result of the `return` command passed in as an argument.

2.6 Other Basic Functions

Aside from the functions we have listed so far, there are a number of other operations for working with basic data structures and interacting with the operating system.

The following functions work on lists:

- `concat : {a} [a] -> [a] -> [a]` takes two lists and returns the concatenation of the two.
- `head : {a} [a] -> a` returns the first element of a list.
- `tail : {a} [a] -> [a]` returns everything except the first element.
- `length : {a} [a] -> Int` counts the number of elements in a list.
- `null : {a} [a] -> Bool` indicates whether a list is empty (has zero elements).
- `nth : {a} [a] -> Int -> a` returns the element at the given position, with `nth l 0` being equivalent to `head l`.
- `for : {m, a, b} [a] -> (a -> m b) -> m [b]` takes a list and a function that runs in some command context. The passed command will be called once for every element of the list, in order. Returns a list of all of the results produced by the command.

For interacting with the operating system, we have:

- `get_opt : Int -> String` returns the command-line argument to `saw` at the given index. Argument 0 is always the name of the `saw` executable itself, and higher indices represent later arguments.

- `exec : String -> [String] -> String -> TopLevel String` runs an external program given, respectively, an executable name, a list of arguments, and a string to send to the standard input of the program. The `exec` command returns the standard output from the program it executes and prints standard error to the screen.
- `exit : Int -> TopLevel ()` stops execution of the current script and returns the given exit code to the operating system.

Finally, there are a few miscellaneous functions and commands:

- `show : {a} a -> String` computes the textual representation of its argument in the same way as `print`, but instead of displaying the value it returns it as a `String` value for later use in the program. This can be useful for constructing more detailed messages later.
- `str_concat : String -> String -> String` concatenates two `String` values, and can also be useful with `show`.
- `time : {a} TopLevel a -> TopLevel a` runs any other `TopLevel` command and prints out the time it took to execute.
- `with_time : {a} TopLevel a -> TopLevel (Int, a)` returns both the original result of the timed command and the time taken to execute it (in milliseconds), without printing anything in the process.

2.7 REPL Actions

There is an additional class of things that one may type at the REPL for interactive use:

- `:cd` changes the REPL's current directory.
- `:pwd` prints the REPL's current directory.
- `:env` displays the values and types of all currently bound variables, including built-in functions and commands.
- `:search` with one or more types (complex types go in parentheses) searches the currently bound variables, including built-in functions and commands, and prints those that mention all the types cited. You can use _ as a wildcard. Free type variables are treated as pattern constraints; use forall-bound type variables using the `{a}` syntax to search specifically for forall-bound types.
- `:tenv` displays the expansions of all currently defined type aliases, including those that are built in.
- `:type` or `:t` checks and prints the type of an arbitrary SAWScript expression:

```
sawscript> :t show
{a.0} a.0 -> String
```

- `:help` or `:h` prints the help text for a built-in function or command:

```
sawscript> :h show

Description
-----------

    show : {a} a -> String

Convert the value of the given expression to a string.
```

- `:quit` or `:q` exits the program.

2.8 Further built-in functions and commands

SAW contains many built-in operations, referred to as "primitives." These appear in SAWScript as built-in functions and commands. The following sections of the manual will introduce many of these.

2.9 Experimental and deprecated functions and commands

Some of the primitives available in SAW at any given time are experimental. These may be incomplete, unfinished, use at your own risk, etc. The functions and commands associated with these are unavailable by default; they can be made visible with the `enable_experimental` command.

Other primitives are considered deprecated. Some of these, as the *deprecation process* proceeds, are unavailable by default.

They can be made visible with the `enable_deprecated` command.

CHAPTER 3

Invoking SAW

Warning

This section is under construction!

There are three ways to run `saw`. The most common is to provide the name of a SAWScript file to run: `saw proofs.saw`. If you leave the file name off, or give the `-I` option, SAW will start an interactive read-eval-print loop ("REPL"). It is also possible to use the `-B` ("batch") option to pass a file of REPL commands to run. This allows automated use of the REPL's `:`-commands.

See *the REPL reference* for additional details about the `saw` executable and its options.

CHAPTER 4

Cryptol and its Role in SAW

Cryptol is a domain-specific language originally designed for the high-level specification of cryptographic algorithms. It is general enough, however, to describe a wide variety of programs, and is particularly applicable to describing computations that operate on streams of data of some fixed size.

In addition to being integrated into SAW, Cryptol is a standalone language with its own manual (https://cdn.prod.website-files.com/673b407e535dbf3b547179dd/677c422f88a92701db5a834d_ProgrammingCryptol.pdf).

SAW includes deep support for Cryptol, and in fact requires the use of Cryptol for most non-trivial tasks. To fully understand the rest of this manual and to effectively use SAW, you will need to develop at least a rudimentary understanding of Cryptol.

The primary use of Cryptol within SAWScript is to construct values of type `Term`. Although `Term` values can be constructed from various sources, inline Cryptol expressions are the most direct and convenient way to create them.

Specifically, a Cryptol expression can be placed inside double curly braces (`{{` and `}}`), resulting in a value of type `Term`. As a very simple example, there is no built-in integer addition operation in SAWScript. However, we can use Cryptol's built-in integer addition operator within SAWScript as follows:

```
sawscript> let t = {{ 0x22 + 0x33 }}
sawscript> print t
85
sawscript> :type t
Term
```

Although it printed out in the same way as an `Int`, it is important to note that `t` actually has type `Term`. We can see how this term is represented internally, before being evaluated, with the `print_term` function.

```
sawscript> print_term t
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.TCNum 8
      x@3 = Cryptol.PLiteralSeqBool x@2
    }
 in Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool x@2)
      (Cryptol.ecNumber (Cryptol.TCNum 34) x@1 x@3)
      (Cryptol.ecNumber (Cryptol.TCNum 51) x@1 x@3)
```

For the moment, it's not important to understand what this output means. We show it only to clarify that `Term` values have their own internal structure that goes beyond what exists in SAWScript. The internal representation of `Term` values is in a language called SAWCore. The full semantics of SAWCore are beyond the scope of this manual.

The text constructed by `print_term` can also be accessed programmatically (instead of printing to the screen) using the `show_term` function, which returns a `String`. The `show_term` function is not a command, so it executes directly and does not need `<-` to bind its result. Therefore, the following will have the same result as the `print_term` command above:

```
sawscript> let s = show_term t
sawscript> :type s
String
sawscript> print s
<same as above>
```

Numbers are printed in decimal notation by default when printing terms, but the following two commands can change that behavior.

- `set_ascii : Bool -> TopLevel ()`, when passed `true`, makes subsequent `print_term` or `show_term` commands print sequences of bytes as ASCII strings (and doesn't affect printing of anything else).
- `set_base : Int -> TopLevel ()` prints all bit vectors in the given base, which can be between 2 and 36 (inclusive).

A `Term` that represents an integer (any bit vector, as affected by `set_base`) can be translated into a SAWScript `Int` using the `eval_int : Term -> Int` function. This function returns an `Int` if the `Term` can be represented as one, and fails at runtime otherwise.

```
sawscript> print (eval_int t)
85
sawscript> print (eval_int {{ True }})
```

(continues on next page)

(continued from previous page)

```
"eval_int" (<stdin>:1:1):
eval_int: argument is not a finite bitvector
sawscript> print (eval_int {{ [True] }})
1
```

Similarly, values of type `Bit` in Cryptol can be translated into values of type `Bool` in SAWScript using the `eval_bool : Term -> Bool` function:

```
sawscript> let b = {{ True }}
sawscript> print_term b
Prelude.True
sawscript> print (eval_bool b)
true
```

Anything with sequence type in Cryptol can be translated into a list of `Term` values in SAWScript using the `eval_list : Term -> [Term]` function.

```
sawscript> let l = {{ [0x01, 0x02, 0x03] }}
sawscript> print_term l
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.PLiteralSeqBool (Cryptol.TCNum 8)
    }
 in [Cryptol.ecNumber (Cryptol.TCNum 1) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 2) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 3) x@1 x@2]
sawscript> print (eval_list l)
[Cryptol.ecNumber (Cryptol.TCNum 1) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
,Cryptol.ecNumber (Cryptol.TCNum 2) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
,Cryptol.ecNumber (Cryptol.TCNum 3) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))]
```

Finally, a list of `Term` values in SAWScript can be collapsed into a single `Term` with sequence type using the `list_term : [Term] -> Term` function, which is the inverse of `eval_list`.

```
sawscript> let ts = eval_list l
sawscript> let l = list_term ts
sawscript> print_term l
let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.PLiteralSeqBool (Cryptol.TCNum 8)
    }
 in [Cryptol.ecNumber (Cryptol.TCNum 1) x@1 x@2
```

(continues on next page)

(continued from previous page)

```
    ,Cryptol.ecNumber (Cryptol.TCNum 2) x@1 x@2
    ,Cryptol.ecNumber (Cryptol.TCNum 3) x@1 x@2]
```

In addition to being able to extract integer and Boolean values from Cryptol expressions, `Term` values can be injected into Cryptol expressions. When SAWScript evaluates a Cryptol expression between `{{` and `}}` delimiters, it does so with several extra bindings in scope:

- Any variable in scope that has SAWScript type `Bool` is visible in Cryptol expressions as a value of type `Bit`.
- Any variable in scope that has SAWScript type `Int` is visible in Cryptol expressions as a *type variable*. Type variables can be demoted to numeric bit vector values using the backtick (`) operator.
- Any variable in scope that has SAWScript type `Term` is visible in Cryptol expressions as a value with the Cryptol type corresponding to the internal type of the term. The power of this conversion is that the `Term` does not need to have originally been derived from a Cryptol expression.

In addition to these rules, bindings created at the Cryptol level, either from included files or inside Cryptol quoting brackets, are visible only to later Cryptol expressions, and not as SAWScript variables.

To make these rules more concrete, consider the following examples. If we bind a SAWScript `Int`, we can use it as a Cryptol type variable. If we create a `Term` variable that internally has function type, we can apply it to an argument within a Cryptol expression, but not at the SAWScript level:

```
sawscript> let n = 8
sawscript> :type n
Int
sawscript> let {{ f (x : [n]) = x + 1 }}
sawscript> :type {{ f }}
Term
sawscript> :type f

<stdin>:1:1-1:2: unbound variable: "f" (<stdin>:1:1-1:2)
sawscript> print {{ f 2 }}
3
```

If `f` was a binding of a SAWScript variable to a `Term` of function type, we would get a different error:

```
sawscript> let f = {{ \(x : [n]) -> x + 1 }}
sawscript> :type {{ f }}
Term
sawscript> :type f
Term
sawscript> print {{ f 2 }}
```

(continues on next page)

(continued from previous page)

```
3
sawscript> print (f 2)

type mismatch: Int -> t.0 and Term
 at "_" (REPL)
 mismatched type constructors: (->) and Term
```

One subtlety of dealing with `Term`s constructed from Cryptol is that because the Cryptol expressions themselves are type checked by the Cryptol type checker, and because they may make use of other `Term` values already in scope, they are not type checked until the Cryptol brackets are evaluated. So type errors at the Cryptol level may occur at runtime from the SAWScript perspective (though they occur before the Cryptol expressions are run).

So far, we have talked about using Cryptol *value* expressions. However, SAWScript can also work with Cryptol *types*. The most direct way to refer to a Cryptol type is to use type brackets: `{|` and `|}`. Any Cryptol type written between these brackets becomes a `Type` value in SAWScript. Some types in Cryptol are *numeric* (also known as *size*) types, and correspond to non-negative integers. These can be translated into SAWScript integers with the `eval_size` function. For example:

```
sawscript> let {{ type n = 16 }}
sawscript> eval_size {| n |}
16
sawscript> eval_size {| 16 |}
16
```

For non-numeric types, `eval_size` fails at runtime:

```
sawscript> eval_size {| [16] |}

"eval_size" (<stdin>:1:1):
eval_size: not a numeric type
```

In addition to the use of brackets to write Cryptol expressions inline, several built-in functions can extract `Term` values from Cryptol files in other ways. The `import` command at the top level imports all top-level definitions from a Cryptol file and places them in scope within later bracketed expressions. This includes Cryptol `foreign` declarations (https://galoisinc.github.io/cryptol/master/FFI.html). If a Cryptol implementation of a foreign function (https://galoisinc.github.io/cryptol/master/FFI.html#cryptol-implementation-of-foreign-functions) is present, then it will be used as the definition when reasoning about the function. Otherwise, the function will be imported as an opaque constant with no definition.

The `cryptol_load` command behaves similarly, but returns a `CryptolModule` instead. If any `CryptolModule` is in scope, its contents are available qualified with the name of the `CryptolModule` variable. A specific definition can be explicitly extracted from a `CryptolModule` using the `cryptol_extract` command:

- `cryptol_extract : CryptolModule -> String -> TopLevel Term`

CHAPTER 5

Loading Code

The first step in analyzing any code is to load it into the system.

5.1 Loading LLVM

To load LLVM code, simply provide the location of a valid bitcode file to the `llvm_load_module` function.

- `llvm_load_module : String -> TopLevel LLVMModule`

The resulting `LLVMModule` can be passed into the various functions described below to perform analysis of specific LLVM functions.

The LLVM bitcode parser should generally work with LLVM versions between 3.5 and 16.0, though it may be incomplete for some versions. Debug metadata has changed somewhat throughout that version range, so is the most likely case of incompleteness. We aim to support every version after 3.5, however, so report any parsing failures as on GitHub (https://github.com/GaloisInc/saw-script/issues).

5.2 Loading Java

Loading Java code is slightly more complex, because of the more structured nature of Java packages. First, when running `saw`, three flags control where to look for classes:

- The `-b` flag takes the path where the `java` executable lives, which is used to locate the Java standard library classes and add them to the class database. Alternatively, one can put the directory where `java` lives on the *`PATH`*, which SAW will search if `-b` is not set.

- The `-j` flag takes the name of a JAR file as an argument and adds the contents of that file to the class database.
- The `-c` flag takes the name of a directory as an argument and adds all class files found in that directory (and its subdirectories) to the class database. By default, the current directory is included in the class path.

Most Java programs will only require setting the `-b` flag (or the `PATH`), as that is enough to bring in the standard Java libraries. Note that when searching the `PATH`, SAW makes assumptions about where the standard library classes live. These assumptions are likely to hold on JDK 7 or later, but they may not hold on older JDKs on certain operating systems. If you are using an old version of the JDK and SAW is unable to find a standard Java class, you may need to specify the location of the standard classes' JAR file with the `-j` flag (or, alternatively, with the `SAW_JDK_JAR` environment variable).

Once the class path is configured, you can pass the name of a class to the `java_load_class` function.

- `java_load_class : String -> TopLevel JavaClass`

The resulting `JavaClass` can be passed into the various functions described below to perform analysis of specific Java methods.

Java class files from any JDK newer than version 6 should work. However, support for JDK 9 and later is experimental. Verifying code that only uses primitive data types is known to work well, but there are some as-of-yet unresolved issues in verifying code involving classes such as `String`. For more information on these issues, refer to this GitHub issue (https://github.com/GaloisInc/crucible/issues/641).

5.3 Loading MIR

To load a piece of Rust code, first compile it to a MIR JSON file, as described in *this section*, and then provide the location of the JSON file to the `mir_load_module` function:

- `mir_load_module : String -> TopLevel MIRModule`

SAW currently supports Rust code that can be built with a January 23, 2023 Rust nightly (https://static.rust-lang.org/dist/2023-01-23/). If you encounter a Rust feature that SAW does not support, please report it on GitHub (https://github.com/GaloisInc/saw-script/issues).

5.4 Notes on Compiling Code for SAW

SAW will generally be able to load arbitrary LLVM bitcode, JVM bytecode, and MIR JSON files, but several guidelines can help make verification easier or more likely to succeed.

5.4.1 Compiling LLVM

For generating LLVM with `clang`, it can be helpful to:

- Turn on debugging symbols with `-g` so that SAW can find source locations of functions, names of variables, etc.
- Optimize with `-O1` so that the generated bitcode more closely matches the C/C++ source, making the results more comprehensible.
- Use `-fno-threadsafe-statics` to prevent `clang` from emitting unnecessary pthread code.
- Link all relevant bitcode with `llvm-link` (including, *e.g.*, the C++ standard library when analyzing C++ code).

All SAW proofs include side conditions to rule out undefined behavior, and proofs will only succeed if all of these side conditions have been discharged. However the default SAW notion of undefined behavior is with respect to the semantics of LLVM, rather than C or C++. If you want to rule out undefined behavior according to the C or C++ standards, consider compiling your code with `-fsanitize=undefined` or one of the related flags[1] to `clang`.

Generally, you'll also want to use `-fsanitize-trap=undefined`, or one of the related flags, to cause the compiled code to use `llvm.trap` to indicate the presence of undefined behavior. Otherwise, the compiled code will call a separate function, such as `__ubsan_handle_shift_out_of_bounds`, for each type of undefined behavior, and SAW currently does not have built in support for these functions (though you could manually create overrides for them in a verification script).

5.4.2 Compiling Java

For Java, the only compilation flag that tends to be valuable is `-g` to retain information about the names of function arguments and local variables.

5.4.3 Compiling MIR

In order to verify Rust code, SAW analyzes Rust's MIR (mid-level intermediate representation) language. In particular, SAW analyzes a particular form of MIR that the `mir-json` (https://github.com/GaloisInc/mir-json) tool produces. You will need to intall `mir-json` and run it on Rust code in order to produce MIR JSON files that SAW can load (see *this section*). You will also need to use `mir-json` to build custom versions of the Rust standard libraries that are more suited to verification purposes.

If you are working from a checkout of the `saw-script` repo, you can install the `mir-json` tool and the custom Rust standard libraries by performing the following steps:

1. Clone the `crucible` (https://github.com/GaloisInc/crucible) and `mir-json` submodules like so:

[1] https://clang.llvm.org/docs/UsersManual.html#controlling-code-generation

```
$ git submodule update deps/crucible deps/mir-json
```

2. Navigate to the `mir-json` submodule:

```
$ cd deps/mir-json
```

3. Follow the instructions laid out in the `mir-json` installation instructions (https://github.com/GaloisInc/mir-json#installation-instructions) in order to install `mir-json`.

4. Run the `mir-json-translate-libs` script in the `mir-json` submodule:

```
$ mir-json-translate-libs
```

   This will compile the custom versions of the Rust standard libraries using `mir-json`, placing the results under the `rlibs` subdirectory.

5. Finally, define a `SAW_RUST_LIBRARY_PATH` environment variable that points to the newly created `rlibs` subdirectory:

```
$ export SAW_RUST_LIBRARY_PATH=<...>/mir-json/rlibs
```

For `cargo`-based projects, `mir-json` provides a `cargo` subcommand called `cargo saw-build` that builds a JSON file suitable for use with SAW. `cargo saw-build` integrates directly with `cargo`, so you can pass flags to it like any other `cargo` subcommand. For example:

```
# Make sure that SAW_RUST_LIBRARY_PATH is defined, as described above
$ cargo saw-build <other cargo flags>
<snip>
linking 11 mir files into <...>/example-364cf2df365c7055.linked-mir.json
<snip>
```

Note that:

- The full output of `cargo saw-build` here is omitted. The important part is the `.linked-mir.json` file that appears after `linking X mir files into`, as that is the JSON file that must be loaded with SAW.
- `SAW_RUST_LIBRARY_PATH` should point to the the MIR JSON files for the Rust standard library.

`mir-json` also supports compiling individual `.rs` files through `mir-json`'s `saw-rustc` command. As the name suggests, it accepts all of the flags that `rustc` accepts. For example:

```
# Make sure that SAW_RUST_LIBRARY_PATH is defined, as described above
$ saw-rustc example.rs <other rustc flags>
<snip>
```

(continues on next page)

(continued from previous page)

```
linking 11 mir files into <...>/example.linked-mir.json
<snip>
```

5.5 Direct Extraction

In many simple cases (such as the mathematical `max` function), the relevant inputs and outputs are immediately apparent. The function takes two integer arguments, always uses both of them, and returns a single integer value, making no other changes to the program state.

In cases like this, a direct translation is possible, given only an identification of which code to execute. Three functions exist to handle such simple code, one for each Crucible backend:

- `llvm_extract : LLVMModule -> String -> TopLevel Term`
- `jvm_extract : JavaClass -> String -> TopLevel Term`
- `mir_extract : MIRModule -> String -> TopLevel Term`

The structure of these extraction functions is essentially identical. The first argument describes where to look for code (in an LLVM module, Java class, or MIR module, loaded as described in the previous section). The second argument is the name of the method or function to extract.

When the extraction functions complete, they return a `Term` corresponding to the value returned by the function or method as a function of its arguments.

These functions currently work only for code that has specific argument and result types:

- For `llvm_extract`, the extracted function must take some fixed number of integral parameters and return an integral result.
- For `jvm_extract`, the extracted function's argument and result types must be scalar types (i.e., not classes or arrays).
- For `mir_extract`, the extracted function's argument and result types must be a primitive integer type (e.g., `u8` or `i8`), a `bool`, a `char`, an array, or a tuple.

Although it is disallowed to extract functions that use pointers, classes, or references in the extracted function's type signature, the implementation of the extracted function is allowed to allocate memory during execution. Also note the following requirements for interacting with global variables:

- For `llvm_extract`, the extracted function is allowed to read from immutable global variables during execution, but it is not allowed to read or write from mutable global variables during execution.
- For `jvm_extract`, the extracted function is allowed to read from or write to any class field or static field (regardless of mutability) during execution. The class and static fields will be given their initial values during extraction (unless they are overwritten during execution).

- For `mir_extract`, the extracted function is allowed to read from immutable static items during execution, and it is allowed to write to mutable static items during execution. The extracted function is not allowed to read from a mutable static item during execution unless the function has written another value to the static item earlier during execution.

5.6 Notes on C++ Analysis

The distance between C++ code and LLVM is greater than between C and LLVM, so some additional considerations come into play when analyzing C++ code with SAW.

The first key issue is that the C++ standard library is large and complex, and tends to be widely used by C++ applications. To analyze most C++ code, it will be necessary to link your code with a version of the `libc++` library[2] compiled to LLVM bitcode. The `wllvm` program can[3] be useful for this.

The C++ standard library includes a number of key global variables, and any code that touches them will require that they be initialized using `llvm_alloc_global`.

Many C++ names are slightly awkward to deal with in SAW. They may be mangled relative to the text that appears in the C++ source code. SAW currently only understands the mangled names. The `llvm-nm` program can be used to show the list of symbols in an LLVM bitcode file, and the `c++filt` program can be used to demangle them, which can help in identifying the symbol you want to refer to. In addition, C++ names from namespaces can sometimes include quote marks in their LLVM encoding. For example:

```
%"class.quux::Foo" = type { i32, i32 }
```

This can be mentioned in SAW by saying:

```
llvm_type "%\"class.quux::Foo\""
```

Finally, there is no support for calling constructors in specifications, so you will need to construct objects piece-by-piece using, *e.g.*, `llvm_alloc` and `llvm_points_to`.

---

[2] https://libcxx.llvm.org/index.html
[3] https://github.com/travitch/whole-program-llvm

CHAPTER 6

Analyzing Hardware Circuits using Yosys

SAW has experimental support for analysis of hardware descriptions written in VHDL (via GHDL (https://github.com/ghdl/ghdl-yosys-plugin)) through an intermediate representation produced by Yosys (https://yosyshq.net/yosys/). This generally follows the same conventions and idioms used in the rest of SAWScript.

6.1 Processing VHDL With Yosys

Given a VHDL file `test.vhd` containing an entity `test`, one can generate an intermediate representation `test.json` suitable for loading into SAW:

```
$ ghdl -a test.vhd
$ yosys
...
Yosys 0.10+1 (git sha1 7a7df9a3b4, gcc 10.3.0 -fPIC -Os)
yosys> ghdl test

1. Executing GHDL.
Importing module test.

yosys> write_json test.json

2. Executing JSON backend.
```

It can sometimes be helpful to invoke additional Yosys passes between the `ghdl` and `write_json` commands. For example, at present SAW does not support the `$pmux` cell type. Yosys is able to

convert `$pmux` cells into trees of `$mux` cells using the `pmuxtree` command. We expect there are many other situations where Yosys' considerable library of commands is valuable for pre-processing.

6.2 Example: Ripple-Carry Adder

Consider three VHDL entities. First, a half-adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity half is
  port (
    a : in std_logic;
    b : in std_logic;
    c : out std_logic;
    s : out std_logic
  );
end half;

architecture halfarch of half is
begin
  c <= a and b;
  s <= a xor b;
end halfarch;
```

Next, a one-bit adder built atop that half-adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity full is
  port (
    a : in std_logic;
    b : in std_logic;
    cin : in std_logic;
    cout : out std_logic;
    s : out std_logic
  );
end full;

architecture fullarch of full is
  signal half0c : std_logic;
  signal half0s : std_logic;
  signal half1c : std_logic;
```

(continues on next page)

(continued from previous page)

```
begin
  half0 : entity work.half port map (a => a, b => b, c => half0c, s =>␣
↪half0s);
  half1 : entity work.half port map (a => half0s, b => cin, c => half1c,
↪ s => s);
  cout <= half0c or half1c;
end fullarch;
```

Finally, a four-bit adder:

```
library ieee;
use ieee.std_logic_1164.all;

entity add4 is
  port (
    a : in std_logic_vector(3 downto 0);
    b : in std_logic_vector(3 downto 0);
    res : out std_logic_vector(3 downto 0)
  );
end add4;

architecture add4arch of add4 is
  signal full0cout : std_logic;
  signal full1cout : std_logic;
  signal full2cout : std_logic;
  signal ignore : std_logic;
begin
  full0 : entity work.full port map (a => a(0), b => b(0), cin => '0',␣
↪cout => full0cout, s => res(0));
  full1 : entity work.full port map (a => a(1), b => b(1), cin =>␣
↪full0cout, cout => full1cout, s => res(1));
  full2 : entity work.full port map (a => a(2), b => b(2), cin =>␣
↪full1cout, cout => full2cout, s => res(2));
  full3 : entity work.full port map (a => a(3), b => b(3), cin =>␣
↪full2cout, cout => ignore, s => res(3));
end add4arch;
```

Using GHDL and Yosys, we can convert the VHDL source above into a format that SAW can import. If all of the code above is in a file `adder.vhd`, we can run the following commands:

```
$ ghdl -a adder.vhd
$ yosys -p 'ghdl add4; write_json adder.json'
```

Side Note: this assumes the ghdl plugin was properly installed, see

https://github.com/ghdl/ghdl-yosys-plugin for installation details.

The produced file `adder.json` can then be loaded into SAW with `yosys_import`.

> **Side Note:** different versions of Yosys and GHDL may produce slightly different naming for the inner modules. TabbyCAD may produce different inner module names as well, so you would need to adjust the script below accordingly to account for different module names.
>
> The generated module names are not guaranteed to be valid Cryptol field names, so it is a good idea to sanity check your JSON file before loading, and at least removing parenthesis from the names with sed: `sed -i '' 's/)//g' adder.json && sed -i '' 's/(//g' adder.json` This preprocessing is needed until #2841 (https://github.com/GaloisInc/saw-script/issues/2841) is fixed.

```
$ saw
...
sawscript> enable_experimental
sawscript> m <- yosys_import "adder.json"
sawscript> :type m
Term
sawscript> print (type m)
[23:57:14.492] {add4 : {a : [4], b : [4]} -> {res : [4]},
 full_Bfullarch : {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
↪,
 half_Bhalfarch : {a : [1], b : [1]} -> {c : [1], s : [1]}}
```

`yosys_import` returns a `Term` with a Cryptol record type, where the fields correspond to each VHDL module. We can access the fields of this record like we would any Cryptol record, and call the functions within like any Cryptol function.

```
sawscript> print (type {{ m.add4 }})
[00:00:25.255] {a : [4], b : [4]} -> {res : [4]}
sawscript> print (eval_int {{ (m.add4 { a = 1, b = 2 }).res }})
[00:02:07.329] 3
```

We can also use all of SAW's infrastructure for asking solvers about `Term`s, such as the `sat` and `prove` commands. For example:

```
sawscript> sat w4 {{ m.add4 === \_ -> { res = 5 } }}
[00:04:41.993] Sat: [_ = (5, 0)]
sawscript> prove z3 {{ m.add4 === \inp -> { res = inp.a + inp.b } }}
[00:05:43.659] Valid
sawscript> prove yices {{ m.add4 === \inp -> { res = inp.a - inp.b } }}
[00:05:56.171] Invalid: [_ = (8, 13)]
```

The full library of `ProofScript` tactics is available in this setting. If necessary, proof tactics like

`simplify` can be used to rewrite goals before querying a solver.

Special support is provided for the common case of equivalence proofs between HDL modules and other `Term`s (e.g. Cryptol functions, other HDL modules, or "extracted" imperative LLVM or JVM code). The command `yosys_verify` has an interface similar to `llvm_verify`: given a specification, some lemmas, and a proof tactic, it produces evidence of a proven equivalence that may be passed as a lemma to future calls of `yosys_verify`. For example, consider the following Cryptol specifications for one-bit and four-bit adders:

```
cryfull :  {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
cryfull inp = { cout = [cout], s = [s] }
  where [cout, s] = zext inp.a + zext inp.b + zext inp.cin

cryadd4 : {a : [4], b : [4]} -> {res : [4]}
cryadd4 inp = { res = inp.a + inp.b }
```

If the Cryptol code above is in a file named `adder.cry`, we can prove equivalence between `cryfull` and the VHDL `full` module:

```
sawscript> import "adder.cry"
sawscript> full_spec <- yosys_verify {{ m.full }} [] {{ cryfull }} []␣
↪w4;
```

The result `full_spec` can then be used as an "override" when proving equivalence between `cryadd4` and the VHDL `add4` module:

```
sawscript> add4_spec <- yosys_verify {{ m.add4 }} [] {{ cryadd4 }}␣
↪[full_spec] w4;
```

The above could also be accomplished through the use of `prove_print` and term rewriting, but it is much more verbose.

`yosys_verify` may also be given a list of preconditions under which the equivalence holds. For example, consider the following Cryptol specification for `full` that ignores the `cin` bit (save them into a `cryfullnocarry.cry` file):

```
cryfullnocarry :  {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
cryfullnocarry inp = { cout = [cout], s = [s] }
  where [cout, s] = zext inp.a + zext inp.b
```

This is not equivalent to `full` in general, but it is if constrained to inputs where `cin = 0`. We may express that precondition like so:

```
sawscript> import "cryfullnocarry.cry"
sawscript> full_nocarry_spec <- yosys_verify {{ m.full_Bfullarch }} [{{\
↪(inp : {a : [1], b : [1], cin : [1]}) -> inp.cin == 0}}] {{␣
```

(continues on next page)

(continued from previous page)

```
↪cryfullnocarry }} [] w4;
```

The resulting override `full_nocarry_spec` may still be used in the proof for `add4` (this is accomplished by rewriting to a conditional expression).

6.3 API Reference

The following commands must first be enabled using `enable_experimental`.

- `yosys_import : String -> TopLevel Term` produces a `Term` given the path to a JSON file produced by the Yosys `write_json` command. The resulting term is a Cryptol record, where each field corresponds to one HDL module exported by Yosys. Each HDL module is in turn represented by a function from a record of input port values to a record of output port values. For example, consider a Yosys JSON file derived from the following VHDL entities:

```
entity half is
  port (
    a : in std_logic;
    b : in std_logic;
    c : out std_logic;
    s : out std_logic
  );
end half;

entity full is
  port (
    a : in std_logic;
    b : in std_logic;
    cin : in std_logic;
    cout : out std_logic;
    s : out std_logic
  );
end full;
```

  The resulting `Term` will have the type:

```
{ half : {a : [1], b : [1]} -> {c : [1], s : [1]}
, full : {a : [1], b : [1], cin : [1]} -> {cout : [1], s : [1]}
}
```

- `yosys_verify : Term -> [Term] -> Term -> [YosysTheorem] -> ProofScript () -> TopLevel YosysTheorem` proves equality between an HDL module and a specification. The first parameter is the HDL module - given a record `m` from `yosys_import`,

this will typically look something like `{{ m.foo }}`. The second parameter is a list of preconditions for the equality. The third parameter is the specification, a term of the same type as the HDL module, which will typically be some Cryptol function or another HDL module. The fourth parameter is a list of "overrides", which witness the results of previous `yosys_verify` proofs. These overrides can be used to simplify terms by replacing use sites of submodules with their specifications.

Note that `Term`s derived from HDL modules are "first class", and are not restricted to `yosys_verify`: they may also be used with SAW's typical `Term` infrastructure like `sat`, `prove_print`, term rewriting, etc. `yosys_verify` simply provides a convenient and familiar interface, similar to `llvm_verify` or `jvm_verify`.

CHAPTER 7

Creating Symbolic Variables

The direct extraction process discussed previously introduces symbolic variables and then abstracts over them, yielding a SAWScript `Term` that reflects the semantics of the original Java, LLVM, or MIR code. For simple functions, this is often the most convenient interface. For more complex code, however, it can be necessary (or more natural) to specifically introduce fresh variables and indicate what portions of the program state they correspond to.

- `fresh_symbolic : String -> Type -> TopLevel Term` is responsible for creating new variables in this context. The first argument is a name used for pretty-printing of terms and counter-examples. In many cases it makes sense for this to be the same as the name used within SAWScript, as in the following:

```
x <- fresh_symbolic "x" ty;
```

However, using the same name is not required.

The second argument to `fresh_symbolic` is the type of the fresh variable. Ultimately, this will be a SAWCore type; however, it is usually convenient to specify it using Cryptol syntax with the type quoting brackets `{|` and `|}`. For example, creating a 32-bit integer, as might be used to represent a Java `int` or an LLVM `i32`, can be done as follows:

```
x <- fresh_symbolic "x" {| [32] |};
```

Although symbolic execution works best on symbolic variables, which are "unbound" or "free", most of the proof infrastructure within SAW uses variables that are *bound* by an enclosing lambda expression. Given a `Term` with free symbolic variables, we can construct a lambda term that binds them in several ways.

- `abstract_symbolic : Term -> Term` finds all symbolic variables in the `Term` and constructs a lambda expression binding each one, in some order. The result is a function of some number of arguments, one for each symbolic variable. It is the simplest but least flexible way to bind symbolic variables.

```
sawscript> x <- fresh_symbolic "x" {| [8] |}
sawscript> let t = {{ x + x }}
sawscript> print_term t
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8))
      x
      x
sawscript> let f = abstract_symbolic t
sawscript> print_term f
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(x : x@1) ->
      Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8)) x x
```

If there are multiple symbolic variables in the `Term` passed to `abstract_symbolic`, the ordering of parameters can be hard to predict. In some cases (such as when a proof is the immediate next step, and it's expected to succeed) the order isn't important. In others, it's nice to have more control over the order.

- `lambda : Term -> Term -> Term` is the building block for controlled binding. It takes two terms: the one to transform, and the portion of the term to abstract over. Generally, the first `Term` is one obtained from `fresh_symbolic` and the second is a `Term` that would be passed to `abstract_symbolic`.

```
sawscript> let f = lambda x t
sawscript> print_term f
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(x : x@1) ->
      Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8)) x x
```

- `lambdas : [Term] -> Term -> Term` allows you to list the order in which symbolic variables should be bound. Consider, for example, a `Term` which adds two symbolic variables:

```
sawscript> x1 <- fresh_symbolic "x1" {| [8] |}
sawscript> x2 <- fresh_symbolic "x2" {| [8] |}
sawscript> let t = {{ x1 + x2 }}
sawscript> print_term t
let { x@1 = Prelude.Vec 8 Prelude.Bool
```

(continues on next page)

(continued from previous page)

```
    }
 in Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8))
      x1
      x2
```

We can turn `t` into a function that takes `x1` followed by `x2`:

```
sawscript> let f1 = lambdas [x1, x2] t
sawscript> print_term f1
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(x1 : x@1) ->
      \(x2 : x@1) ->
        Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8)) x1
          x2
```

Or we can turn `t` into a function that takes `x2` followed by `x1`:

```
sawscript> let f1 = lambdas [x2, x1] t
sawscript> print_term f1
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(x2 : x@1) ->
      \(x1 : x@1) ->
        Cryptol.ecPlus x@1 (Cryptol.PArithSeqBool (Cryptol.TCNum 8)) x1
          x2
```

CHAPTER 8

Symbolic Execution

Analysis of Java and LLVM within SAWScript relies heavily on *symbolic execution*, so some background on how this process works can help with understanding the behavior of the available built-in functions.

At the most abstract level, symbolic execution works like normal program execution except that the values of all variables within the program can be arbitrary *expressions*, potentially containing free variables, rather than concrete values. Therefore, each symbolic execution corresponds to some set of possible concrete executions.

As a concrete example, consider the following C program that returns the maximum of two values:

```
unsigned int max(unsigned int x, unsigned int y) {
    if (y > x) {
        return y;
    } else {
        return x;
    }
}
```

If you call this function with two concrete inputs, like this:

```
int r = max(5, 4);
```

then it will assign the value `5` to `r`. However, we can also consider what it will do for *arbitrary* inputs. Consider the following example:

```
int r = max(a, b);
```

where `a` and `b` are variables with unknown values. It is still possible to describe the result of the `max` function in terms of `a` and `b`. The following expression describes the value of `r`:

```
ite (b > a) b a
```

where `ite` is the "if-then-else" mathematical function, which based on the value of the first argument returns either the second or third. One subtlety of constructing this expression, however, is the treatment of conditionals in the original program. For any concrete values of `a` and `b`, only one branch of the `if` statement will execute. During symbolic execution, on the other hand, it is necessary to execute *both* branches, track two different program states (each composed of symbolic values), and then *merge* those states after executing the `if` statement. This merging process takes into account the original branch condition and introduces the `ite` expression.

A symbolic execution system, then, is very similar to an interpreter that has a different notion of what constitutes a value and executes *all* paths through the program instead of just one. Therefore, the execution process is similar to that of a normal interpreter, and the process of generating a model for a piece of code is similar to building a test harness for that same code.

More specifically, the setup process for a test harness typically takes the following form:

1. Initialize or allocate any resources needed by the code. For Java and LLVM code, this typically means allocating memory and setting the initial values of variables.
2. Execute the code.
3. Check the desired properties of the system state after the code completes.

Accordingly, three pieces of information are particularly relevant to the symbolic execution process, and are therefore needed as input to the symbolic execution system:

- The initial (potentially symbolic) state of the system.
- The code to execute.
- The final state of the system, and which parts of it are relevant to the properties being tested.

In the following sections, we describe how the Java and LLVM analysis primitives work in the context of these key concepts. We start with the simplest situation, in which the structure of the initial and final states can be directly inferred, and move on to more complex cases that require more information from the user.

CHAPTER 9

Symbolic Termination

Above we described the process of executing multiple branches and merging the results when encountering a conditional statement in the program. When a program contains loops, the branch that chooses to continue or terminate a loop could go either way. Therefore, without a bit more information, the most obvious implementation of symbolic execution would never terminate when executing programs that contain loops.

The solution to this problem is to analyze the branch condition whenever considering multiple branches. If the condition for one branch can never be true in the context of the current symbolic state, there is no reason to execute that branch, and skipping it can make it possible for symbolic execution to terminate.

Directly comparing the branch condition to a constant can sometimes be enough to ensure termination. For example, in simple, bounded loops like the following, comparison with a constant is sufficient.

```
for (int i = 0; i < 10; i++) {
    // do something
}
```

In this case, the value of `i` is always concrete, and will eventually reach the value `10`, at which point the branch corresponding to continuing the loop will be infeasible.

As a more complex example, consider the following function:

```
uint8_t f(uint8_t i) {
  int done = 0;
  while (!done) {
```

(continues on next page)

(continued from previous page)

```
    if (i % 8 == 0) done = 1;
    i += 5;
  }
  return i;
}
```

The loop in this function can only be determined to symbolically terminate if the analysis takes into account algebraic rules about common multiples. Similarly, it can be difficult to prove that a base case is eventually reached for all inputs to a recursive program.

In this particular case, however, the code *is* guaranteed to terminate after a fixed number of iterations (where the number of possible iterations is a function of the number of bits in the integers being used). To show that the last iteration is in fact the last possible one, it's necessary to do more than just compare the branch condition with a constant. Instead, we can use the same proof tools that we use to ultimately analyze the generated models to, early in the process, prove that certain branch conditions can never be true (i.e., are *unsatisfiable*).

Normally, most of the Java and LLVM analysis commands simply compare branch conditions to the constant `True` or `False` to determine whether a branch may be feasible. However, each form of analysis allows branch satisfiability checking to be turned on if needed, in which case functions like `f` above will terminate.

Next, we examine the details of the specific commands available to analyze JVM and LLVM programs.

CHAPTER 10

The Term Type

Perhaps the most important type in SAWScript, and the one most unlike the built-in types of most other languages, is the `Term` type. Essentially, a value of type `Term` precisely describes all possible computations performed by some program. In particular, if two `Term` values are *equivalent*, then the programs that they represent will always compute the same results given the same inputs. We will say more later about exactly what it means for two terms to be equivalent, and how to determine whether two terms are equivalent.

Before exploring the `Term` type more deeply, it is important to understand the role of the Cryptol language in SAW – make sure to read *that section of the manual* before continuing.

CHAPTER 11

Specification-Based Verification

The built-in functions described so far work by extracting models of code that can then be used for a variety of purposes, including proofs about the properties of the code.

When the goal is to prove equivalence between some LLVM, Java, or MIR code and a specification, however, a more declarative approach is sometimes convenient. The following sections describe an approach that combines model extraction and verification with respect to a specification. A verified specification can then be used as input to future verifications, allowing the proof process to be decomposed.

11.1 Running a Verification

Verification of LLVM is controlled by the `llvm_verify` command.

```
llvm_verify :
  LLVMModule ->
  String ->
  [CrucibleMethodSpec] ->
  Bool ->
  LLVMSetup () ->
  ProofScript SatResult ->
  TopLevel CrucibleMethodSpec
```

The first two arguments specify the module and function name to verify, as with `llvm_verify`. The third argument specifies the list of already-verified specifications to use for compositional verification (described later; use `[]` for now). The fourth argument specifies whether to do path satisfiability

checking, and the fifth gives the specification of the function to be verified. Finally, the last argument gives the proof script to use for verification. The result is a proved specification that can be used to simplify verification of functions that call this one.

Similar commands are available for JVM programs:

```
jvm_verify :
  JavaClass ->
  String ->
  [JVMMethodSpec] ->
  Bool ->
  JVMSetup () ->
  ProofScript SatResult ->
  TopLevel JVMMethodSpec
```

And for MIR programs:

```
mir_verify :
  MIRModule ->
  String ->
  [MIRSpec] ->
  Bool ->
  MIRSetup () ->
  ProofScript () ->
  TopLevel MIRSpec
```

11.1.1 Running a MIR-based verification

The `String` supplied as an argument to `mir_verify` is expected to be a function *identifier*. An identifier is expected adhere to one of the following conventions:

- `<crate name>/<disambiguator>::<function path>`
- `<crate name>::<function path>`

Where:

- `<crate name>` is the name of the crate in which the function is defined. (If you produced your MIR JSON file by compiling a single `.rs` file with `saw-rustc`, then the crate name is the same as the name of the file, but without the `.rs` file extension.)
- `<disambiguator>` is a hash of the crate and its dependencies. In extreme cases, it is possible for two different crates to have identical crate names, in which case the disambiguator must be used to distinguish between the two crates. In the common case, however, most crate names will correspond to exactly one disambiguator, and you are allowed to leave out the `/<disambiguator>` part of the `String` in this case. If you supply an identifier with an ambiguous crate name and omit the disambiguator, then SAW will raise an error.

- `<function path>` is the path to the function within the crate. Sometimes, this is as simple as the function name itself. In other cases, a function path may involve multiple *segments*, depending on the module hierarchy for the program being verified. For instance, a `read` function located in `core/src/ptr/mod.rs` will have the identifier:

  ```
  core::ptr::read
  ```

  Where `core` is the crate name and `ptr::read` is the function path, which has two segments `ptr` and `read`. There are also some special forms of segments that appear for functions defined in certain language constructs. For instance, if a function is defined in an `impl` block, then it will have `{impl}` as one of its segments, e.g.,

  ```
  core::ptr::const_ptr::{impl}::offset
  ```

  If you are in doubt about what the full identifier for a given function is, consult the MIR JSON file for your program.

  The Rust compiler generates a separate instance for each use of a polymorphic function at a different type. These instances have a compiler-generated name, so the easiest way to refer to them is by using the command `mir_find_name : MIRModule -> String -> [MIRType] -> String`. Given a Rust module, the name of a polymorphic function, and a list of types, `mir_find_name` will return the name of the corresponding instantiation. It throws an exception if the polymorphic function or the given instantiation are not found.

Now we describe how to construct a value of type `LLVMSetup ()`, `JVMSetup ()`, or `MIRSetup ()`.

11.2 Structure of a Specification

A specifications for Crucible consists of three logical components:

- A specification of the initial state before execution of the function.
- A description of how to call the function within that state.
- A specification of the expected final value of the program state.

These three portions of the specification are written in sequence within a `do` block of type `{LLVM, JVM,MIR}Setup`. The command `{llvm,jvm,mir}_execute_func` separates the specification of the initial state from the specification of the final state, and specifies the arguments to the function in terms of the initial state. Most of the commands available for state description will work either before or after `{llvm,jvm,mir}_execute_func`, though with slightly different meaning, as described below.

> **Note**
>
> Every `{LLVM,JVM,MIR}Setup` block **must contain exactly one** `{llvm,jvm,mir}_execute_func` invocation. If it is omitted, SAW will reject the specification.

11.3 Creating Fresh Variables

In any case where you want to prove a property of a function for an entire class of inputs (perhaps all inputs) rather than concrete values, the initial values of at least some elements of the program state must contain fresh variables. These are created in a specification with the `{llvm,jvm,mir}_fresh_var` commands rather than `fresh_symbolic`.

- `llvm_fresh_var : String -> LLVMType -> LLVMSetup Term`
- `jvm_fresh_var : String -> JavaType -> JVMSetup Term`
- `mir_fresh_var : String -> MIRType -> MIRSetup Term`

The first parameter to both functions is a name, used only for presentation. It's possible (though not recommended) to create multiple variables with the same name, but SAW will distinguish between them internally. The second parameter is the LLVM, Java, or MIR type of the variable. The resulting `Term` can be used in various subsequent commands.

Note that the second parameter to `{llvm,jvm,mir}_fresh_var` must be a type that has a counterpart in Cryptol. (For more information on this, refer to the "Cryptol type correspondence" section.) If the type does not have a Cryptol counterpart, the function will raise an error. If you do need to create a fresh value of a type that cannot be represented in Cryptol, consider using a function such as `llvm_fresh_expanded_val` (for LLVM verification) or `mir_fresh_expanded_value` (for MIR verification).

LLVM types are built with this set of functions:

- `llvm_int : Int -> LLVMType`
- `llvm_alias : String -> LLVMType`
- `llvm_array : Int -> LLVMType -> LLVMType`
- `llvm_float : LLVMType`
- `llvm_double : LLVMType`
- `llvm_packed_struct : [LLVMType] -> LLVMType`
- `llvm_struct_type : [LLVMType] -> LLVMType`

Java types are built up using the following functions:

- `java_bool : JavaType`
- `java_byte : JavaType`

- `java_char : JavaType`
- `java_short : JavaType`
- `java_int : JavaType`
- `java_long : JavaType`
- `java_float : JavaType`
- `java_double : JavaType`
- `java_class : String -> JavaType`
- `java_array : Int -> JavaType -> JavaType`

MIR types are built up using the following functions:

- `mir_adt : MIRAdt -> MIRType`
- `mir_array : Int -> MIRType -> MIRType`
- `mir_bool : MIRType`
- `mir_char : MIRType`
- `mir_i8 : MIRType`
- `mir_i6 : MIRType`
- `mir_i32 : MIRType`
- `mir_i64 : MIRType`
- `mir_i128 : MIRType`
- `mir_isize : MIRType`
- `mir_f32 : MIRType`
- `mir_f64 : MIRType`
- `mir_lifetime : MIRType`
- `mir_raw_ptr_const : MIRType -> MIRType`
- `mir_raw_ptr_mut : MIRType -> MIRType`
- `mir_ref : MIRType -> MIRType`
- `mir_ref_mut : MIRType -> MIRType`
- `mir_slice : MIRType -> MIRType`
- `mir_str : MIRType`
- `mir_tuple : [MIRType] -> MIRType`
- `mir_u8 : MIRType`

- `mir_u6 : MIRType`
- `mir_u32 : MIRType`
- `mir_u64 : MIRType`
- `mir_u128 : MIRType`
- `mir_usize : MIRType`

Most of these types are straightforward mappings to the standard LLVM and Java types. The one key difference is that arrays must have a fixed, concrete size. Therefore, all analysis results are valid only under the assumption that any arrays have the specific size indicated, and may not hold for other sizes.

The `llvm_int` function takes an `Int` parameter indicating the variable's bit width. For example, the C `uint16_t` and `int16_t` types correspond to `llvm_int 16`. The C `bool` type is slightly trickier. A bare `bool` type typically corresponds to `llvm_int 1`, but if a `bool` is a member of a composite type such as a pointer, array, or struct, then it corresponds to `llvm_int 8`. This is due to a peculiarity in the way Clang compiles `bool` down to LLVM. When in doubt about how a `bool` is represented, check the LLVM bitcode by compiling your code with `clang -S -emit-llvm`.

LLVM types can also be specified in LLVM syntax directly by using the `llvm_type` function.

- `llvm_type : String -> LLVMType`

For example, `llvm_type "i32"` yields the same result as `llvm_int 32`.

The most common use for creating fresh variables is to state that a particular function should have the specified behaviour for arbitrary initial values of the variables in question. Sometimes, however, it can be useful to specify that a function returns (or stores, more about this later) an arbitrary value, without specifying what that value should be. To express such a pattern, you can also run `llvm_fresh_var` from the post state (i.e., after `llvm_execute_func`).

11.4 The LLVMValue, JVMValue, and MIRValue Types

Many specifications require reasoning about both pure values and about the configuration of the heap. The "setup value" types (`LLVMValue`, `JVMValue`, and `MIRValue`) correspond to values that can occur during symbolic execution, which includes both `Term` values, pointers, and composite types consisting of either of these (both structures and arrays).

The `llvm_term`, `jvm_term`, and `mir_term` functions create a `LLVMValue`, `JVMValue`, or `MIRValue`, respectively, from a `Term`:

- `llvm_term : Term -> LLVMValue`
- `jvm_term : Term -> JVMValue`
- `mir_term : Term -> MIRValue`

The value that these functions return will have an LLVM, JVM, or MIR type corresponding to the Cryptol type of the `Term` argument. (For more information on this, refer to the "Cryptol type correspondence" section.) If the type does not have a Cryptol counterpart, the function will raise an error.

11.4.1 Cryptol type correspondence

The `{llvm,jvm,mir}_fresh_var` functions take an LLVM, JVM, or MIR type as an argument and produces a `Term` variable of the corresponding Cryptol type as output. Similarly, the `{llvm,jvm,mir}_term` functions take a Cryptol `Term` as input and produce a value of the corresponding LLVM, JVM, or MIR type as output. This section describes precisely which types can be converted to Cryptol types (and vice versa) in this way.

LLVM verification

The following LLVM types correspond to Cryptol types:

- `llvm_alias <name>`: Corresponds to the same Cryptol type as the type used in the definition of `<name>`.
- `llvm_array <n> <ty>`: Corresponds to the Cryptol sequence `[<n>][<cty>]`, where `<cty>` is the Cryptol type corresponding to `<ty>`.
- `llvm_int <n>`: Corresponds to the Cryptol word `[<n>]`.
- `llvm_struct_type [<ty_1>, ..., <ty_n>]` and `llvm_packed_struct [<ty_1>, ..., <ty_n>]`: Corresponds to the Cryptol tuple `(<cty_1>, ..., <cty_n>)`, where `<cty_i>` is the Cryptol type corresponding to `<ty_i>` for each `i` ranging from `1` to `n`.

The following LLVM types do *not* correspond to Cryptol types:

- `llvm_double`
- `llvm_float`
- `llvm_pointer`

JVM verification

The following Java types correspond to Cryptol types:

- `java_array <n> <ty>`: Corresponds to the Cryptol sequence `[<n>][<cty>]`, where `<cty>` is the Cryptol type corresponding to `<ty>`.
- `java_bool`: Corresponds to the Cryptol `Bit` type.
- `java_byte`: Corresponds to the Cryptol `[8]` type.
- `java_char`: Corresponds to the Cryptol `[16]` type.
- `java_int`: Corresponds to the Cryptol `[32]` type.

- `java_long`: Corresponds to the Cryptol `[64]` type.
- `java_short`: Corresponds to the Cryptol `[16]` type.

The following Java types do *not* correspond to Cryptol types:

- `java_class`
- `java_double`
- `java_float`

MIR verification

The following MIR types correspond to Cryptol types:

- `mir_array <n> <ty>`: Corresponds to the Cryptol sequence `[<n>][<cty>]`, where `<cty>` is the Cryptol type corresponding to `<ty>`.
- `mir_bool`: Corresponds to the Cryptol `Bit` type.
- `mir_char`: Corresponds to the Cryptol `[32]` type.
- `mir_i8` and `mir_u8`: Corresponds to the Cryptol `[8]` type.
- `mir_i16` and `mir_u16`: Corresponds to the Cryptol `[16]` type.
- `mir_i32` and `mir_u32`: Corresponds to the Cryptol `[32]` type.
- `mir_i64` and `mir_u64`: Corresponds to the Cryptol `[64]` type.
- `mir_i128` and `mir_u128`: Corresponds to the Cryptol `[128]` type.
- `mir_isize` and `mir_usize`: Corresponds to the Cryptol `[64]` type.
- `mir_tuple [<ty_1>, ..., <ty_n>]`: Corresponds to the Cryptol tuple `(<cty_1>, ..., <cty_n>)`, where `<cty_i>` is the Cryptol type corresponding to `<ty_i>` for each `i` ranging from `1` to `n`.

The following MIR types do *not* correspond to Cryptol types:

- `mir_adt`
- `mir_f32`
- `mir_f64`
- `mir_ref` and `mir_ref_mut`
- `mir_raw_ptr_const` and `mir_raw_ptr_mut`
- `mir_slice`
- `mir_str`

11.5 Executing

Once the initial state has been configured, the `{llvm,jvm,mir}_execute_func` command specifies the parameters of the function being analyzed in terms of the state elements already configured.

- `llvm_execute_func : [LLVMValue] -> LLVMSetup ()`
- `jvm_execute_func : [JVMValue] -> JVMSetup ()`
- `mir_execute_func : [MIRValue] -> MIRSetup ()`

Every `{llvm,jvm,mir}_execute_func` specification block **must contain exactly one** execution command. SAW now rejects any specification that omits the corresponding `{llvm,jvm,mir}_execute_func`.

11.6 Return Values

To specify the value that should be returned by the function being verified use the `{llvm,jvm,mir}_return` command.

- `llvm_return : LLVMValue -> LLVMSetup ()`
- `jvm_return : JVMValue -> JVMSetup ()`
- `mir_return : MIRValue -> MIRSetup ()`

The rules for `{llvm,jvm,mir}_return` depend on the backend and the function's declared return type, as summarized below:

Backend	Function return type	Is `{…}_return` required?	Is `{…}_return` allowed?
LLVM	non-`void`	Yes	Yes
LLVM	`void`	No	No
JVM	non-`void`	Yes	Yes
JVM	`void`	No	No
MIR	non-`()`	Yes	Yes
MIR	`()`	No	Yes

When the specification should not constrain the function's return value, introduce a fresh variable in the post-state and return it unchanged:

```
do {
  ...
  {llvm,jvm,mir}_execute_func <args>;
  ...
  ret <- {llvm,jvm,mir}_fresh_var "ret" <type>;
```

(continues on next page)

(continued from previous page)

```
  {llvm,jvm,mir}_return ({llvm,jvm,mir}_term ret);
}
```

This pattern asserts only that the function returns a value of the correct type, without placing any further requirements on what that value must be.

11.7 A First Simple Example (Revisited)

Warning

This section is under construction!

See *the example's introduction*.

11.8 Compositional Verification

The primary advantage of the specification-based approach to verification is that it allows for compositional reasoning. That is, when proving properties of a given method or function, we can make use of properties we have already proved about its callees rather than analyzing them anew. This enables us to reason about much larger and more complex systems than otherwise possible.

The `llvm_verify`, `jvm_verify`, and `mir_verify` functions return values of type `CrucibleMethodSpec`, `JVMMethodSpec`, and `MIRMethodSpec`, respectively. These values are opaque objects that internally contain both the information provided in the associated `LLVMSetup`, `JVMSetup`, or `MIRSetup` blocks, respectively, and the results of the verification process.

Any of these `MethodSpec` objects can be passed in via the third argument of the `..._verify` functions. For any function or method specified by one of these parameters, the simulator will not follow calls to the associated target. Instead, it will perform the following steps:

- Check that all `llvm_points_to` and `llvm_precond` statements (or the corresponding JVM or MIR statements) in the specification are satisfied.
- Update the simulator state and optionally construct a return value as described in the specification.

More concretely, building on the previous example, say we have a doubling function written in terms of `add`:

```
uint32_t dbl(uint32_t x) {
    return add(x, x);
}
```

It has a similar specification to `add`:

```
let dbl_setup = do {
    x <- llvm_fresh_var "x" (llvm_int 32);
    llvm_execute_func [llvm_term x];
    llvm_return (llvm_term {{ x + x : [32] }});
};
```

And we can verify it using what we've already proved about `add`:

```
llvm_verify m "dbl" [add_ms] false dbl_setup abc;
```

In this case, doing the verification compositionally doesn't save computational effort, since the functions are so simple, but it illustrates the approach.

11.8.1 Compositional Verification and Mutable Allocations

A common pitfall when using compositional verification is to reuse a specification that underspecifies the value of a mutable allocation. In general, doing so can lead to unsound verification, so SAW goes through great lengths to check for this.

Here is an example of this pitfall in an LLVM verification. Given this C code:

void side_effect(uint32_t *a) { *a = 0; }

uint32_t foo(uint32_t x) { uint32_t b = x; side_effect(&b); return b; }

And the following SAW specifications:

```
let side_effect_spec = do {
  a_ptr <- llvm_alloc (llvm_int 32);
  a_val <- llvm_fresh_var "a_val" (llvm_int 32);
  llvm_points_to a_ptr (llvm_term a_val);

  llvm_execute_func [a_ptr];
};

let foo_spec = do {
  x <- llvm_fresh_var "x" (llvm_int 32);

  llvm_execute_func [llvm_term x];

  llvm_return (llvm_term x);
};
```

Should SAW be able to verify the `foo` function against `foo_spec` using compositional verification? That is, should the following be expected to work?

```
side_effect_ov <- llvm_verify m "side_effect" [] false side_effect_spec␣
↪z3;
llvm_verify m "foo" [side_effect_ov] false foo_spec z3;
```

A literal reading of `side_effect_spec` would suggest that the `side_effect` function allocates `a_ptr` but then does nothing with it, implying that `foo` returns its argument unchanged. This is incorrect, however, as the `side_effect` function actually changes its argument to point to `0`, so the `foo` function ought to return `0` as a result. SAW should not verify `foo` against `foo_spec`, and indeed it does not.

The problem is that `side_effect_spec` underspecifies the value of `a_ptr` in its postconditions, which can lead to the potential unsoundness seen above when `side_effect_spec` is used in compositional verification. To prevent this source of unsoundness, SAW will *invalidate* the underlying memory of any mutable pointers (i.e., those declared with `llvm_alloc`, not `llvm_alloc_global`) allocated in the preconditions of compositional override that do not have a corresponding `llvm_points_to` statement in the postconditions. Attempting to read from invalidated memory constitutes an error, as can be seen in this portion of the error message when attempting to verify `foo` against `foo_spec`:

```
invalidate (state of memory allocated in precondition (at side.
↪saw:3:12) not described in postcondition)
```

To fix this particular issue, add an `llvm_points_to` statement to `side_effect_spec`:

```
let side_effect_spec = do {
  a_ptr <- llvm_alloc (llvm_int 32);
  a_val <- llvm_fresh_var "a_val" (llvm_int 32);
  llvm_points_to a_ptr (llvm_term a_val);

  llvm_execute_func [a_ptr];

  // This is new
  llvm_points_to a_ptr (llvm_term {{ 0 : [32] }});
};
```

After making this change, SAW will reject `foo_spec` for a different reason, as it claims that `foo` returns its argument unchanged when it actually returns `0`.

Note that invalidating memory itself does not constitute an error, so if the `foo` function never read the value of `b` after calling `side_effect(&b)`, then there would be no issue. It is only when a function attempts to *read* from invalidated memory that an error is thrown. In general, it can be difficult to predict when a function will or will not read from invalidated memory, however. For this reason, it is recommended to always specify the values of mutable allocations in the postconditions of your specs, as it can avoid pitfalls like the one above.

The same pitfalls apply to compositional MIR verification, with a couple of key differences. In MIR

verification, mutable references are allocated using `mir_alloc_mut`. Here is a Rust version of the pitfall program above:

```
pub fn side_effect(a: &mut u32) {
    *a = 0;
}

pub fn foo(x: u32) -> u32 {
    let mut b: u32 = x;
    side_effect(&mut b);
    b
}
```

```
let side_effect_spec = do {
  a_ref <- mir_alloc_mut mir_u32;
  a_val <- mir_fresh_var "a_val" mir_u32;
  mir_points_to a_ref (mir_term a_val);

  mir_execute_func [a_ref];
};

let foo_spec = do {
  x <- mir_fresh_var "x" mir_u32;

  mir_execute_func [mir_term x];

  mir_return (mir_term {{ x }});
};
```

Just like above, if you attempted to prove `foo` against `foo_spec` using compositional verification:

```
side_effect_ov <- mir_verify m "test::side_effect" [] false side_effect_
↪spec z3;
mir_verify m "test::foo" [side_effect_ov] false foo_spec z3;
```

Then SAW would throw an error, as `side_effect_spec` underspecifies the value of `a_ref` in its postconditions. `side_effect_spec` can similarly be repaired by adding a `mir_points_to` statement involving `a_ref` in `side_effect_spec`'s postconditions.

MIR verification differs slightly from LLVM verification in how it catches underspecified mutable allocations when using compositional overrides. The LLVM memory model achieves this by invalidating the underlying memory in underspecified allocations. The MIR memory model, on the other hand, does not have a direct counterpart to memory invalidation. As a result, any MIR overrides must specify the values of all mutable allocations in their postconditions, *even if the function that calls the override never uses the allocations*.

To illustrate this point more finely, suppose that the `foo` function had instead been defined like this:

```
pub fn foo(x: u32) -> u32 {
    let mut b: u32 = x;
    side_effect(&mut b);
    42
}
```

Here, it does not particularly matter what effects the `side_effect` function has on its argument, as `foo` will now return `42` regardless. Still, if you attempt to prove `foo` by using `side_effect` as a compositional override, then it is strictly required that you specify the value of `side_effect`'s argument in its postconditions, even though the answer that `foo` returns is unaffected by this. This is in contrast with LLVM verification, where one could get away without specifying `side_effect`'s argument in this example, as the invalidated memory in `b` would never be read.

11.8.2 Compositional Verification and Mutable Global Variables

Just like with local mutable allocations (see the previous section), specifications used in compositional overrides must specify the values of mutable global variables in their postconditions. To illustrate this using LLVM verification, here is a variant of the C program from the previous example that uses a mutable global variable `a`:

```
uint32_t a = 42;

void side_effect(void) {
  a = 0;
}

uint32_t foo(void) {
  side_effect();
  return a;
}
```

If we attempted to verify `foo` against this `foo_spec` specification using compositional verification:

```
let side_effect_spec = do {
  llvm_alloc_global "a";
  llvm_points_to (llvm_global "a") (llvm_global_initializer "a");

  llvm_execute_func [];
};

let foo_spec = do {
  llvm_alloc_global "a";
  llvm_points_to (llvm_global "a") (llvm_global_initializer "a");
```

(continues on next page)

(continued from previous page)

```
  llvm_execute_func [];

  llvm_return (llvm_global_initializer "a");
};

side_effect_ov <- llvm_verify m "side_effect" [] false side_effect_spec␣
↪z3;
llvm_verify m "foo" [side_effect_ov] false foo_spec z3;
```

Then SAW would reject it, as `side_effect_spec` does not specify what `a`'s value should be in its postconditions. Just as with local mutable allocations, SAW will invalidate the underlying memory in `a`, and subsequently reading from `a` in the `foo` function will throw an error. The solution is to add an `llvm_points_to` statement in the postconditions that declares that `a`'s value is set to `0`.

The same concerns apply to MIR verification, where mutable global variables are referred to as `static mut` items. (See the *MIR static items* section for more information). Here is a Rust version of the program above:

```
static mut A: u32 = 42;

pub fn side_effect() {
    unsafe {
        A = 0;
    }
}

pub fn foo() -> u32 {
    side_effect();
    unsafe { A }
}
```

```
let side_effect_spec = do {
  mir_points_to (mir_static "test::A") (mir_static_initializer "test::A
↪");

  mir_execute_func [];
};

let foo_spec = do {
  mir_points_to (mir_static "test::A") (mir_static_initializer "test::A
↪");
```

(continues on next page)

(continued from previous page)

```
  mir_execute_func [];

  mir_return (mir_static_initializer "test::A");
};

side_effect_ov <- mir_verify m "side_effect" [] false side_effect_spec␣
↪z3;
mir_verify m "foo" [side_effect_ov] false foo_spec z3;
```

Just as above, we can repair this by adding a `mir_points_to` statement in `side_effect_spec`'s postconditions that specifies that `A` is set to `0`.

Recall from the previous section that MIR verification is stricter than LLVM verification when it comes to specifying mutable allocations in the postconditions of compositional overrides. This is especially true for mutable static items. In MIR verification, any compositional overrides must specify the values of all mutable static items in the entire program in their postconditions, *even if the function that calls the override never uses the static items*. For example, if the `foo` function were instead defined like this:

```
pub fn foo() -> u32 {
    side_effect();
    42
}
```

Then it is still required for `side_effect_spec` to specify what `A`'s value will be in its postconditions, despite the fact that this has no effect on the value that `foo` will return.

11.9 Specifying Heap Layout

Most functions that operate on pointers expect that certain pointers point to allocated memory before they are called. The `llvm_alloc` command allows you to specify that a function expects a particular pointer to refer to an allocated region appropriate for a specific type.

- `llvm_alloc : LLVMType -> LLVMSetup LLVMValue`

This command returns a `LLVMValue` consisting of a pointer to the allocated space, which can be used wherever a pointer-valued `LLVMValue` can be used.

In the initial state, `llvm_alloc` specifies that the function expects a pointer to allocated space to exist. In the final state, it specifies that the function itself performs an allocation.

In LLVM, it's also possible to construct fresh pointers that do not point to allocated memory (which can be useful for functions that manipulate pointers but not the values they point to):

- `llvm_fresh_pointer : LLVMType -> LLVMSetup LLVMValue`

The NULL pointer is called `llvm_null` in LLVM and `jvm_null` in JVM:

- `llvm_null : LLVMValue`
- `jvm_null : JVMValue`

One final, slightly more obscure command is the following:

- `llvm_alloc_readonly : LLVMType -> LLVMSetup LLVMValue`

This works like `llvm_alloc` except that writes to the space allocated are forbidden. This can be useful for specifying that a function should take as an argument a pointer to allocated space that it will not modify. Unlike `llvm_alloc`, regions allocated with `llvm_alloc_readonly` are allowed to alias other read-only regions.

In the Java implementation, separate functions exist for specifying that arrays or objects are allocated:

- `jvm_alloc_array : Int -> JavaType -> JVMSetup JVMValue` specifies an array of the given concrete size, with elements of the given type.
- `jvm_alloc_object : String -> JVMSetup JVMValue` specifies an object of the given class name.

The MIR implementation also has a `mir_alloc` function, which behaves similarly to `llvm_alloc`. `mir_alloc` creates an immutable reference, but there is also a `mir_alloc_mut` function for creating a mutable reference:

- `mir_alloc : MIRType -> MIRSetup MIRValue`
- `mir_alloc_mut : MIRType -> MIRSetup MIRValue`

MIR tracks whether references are mutable or immutable at the type level, so it is important to use the right allocation command for a given reference type.

In addition, MIR also has immutable and mutable raw pointers, written in Rust as `*const T` and `*mut T` respectively. As far as SAW is concerned, they behave similarly to references, and they can be created with `mir_alloc_raw_ptr_const` and `mir_alloc_raw_ptr_mut` respectively.

- `mir_alloc_raw_ptr_const : MIRType -> MIRSetup MIRValue`
- `mir_alloc_raw_ptr_mut : MIRType -> MIRSetup MIRValue`

In low-level Rust code, it is possible to get a raw pointer into an allocation of multiple values, do pointer arithmetic on it, and then read from or write to various values within the allocation. The `crucible-mir` memory model keeps track of these allocation sizes to check the validity of these pointer operations. `mir_alloc_raw_ptr_const` and `mir_alloc_raw_ptr_mut` create single-value allocations which don't allow for pointer arithmetic. To model pointers which point to allocations containing multiple values, there are the `mir_alloc_raw_ptr_const_multi` and `mir_alloc_raw_ptr_mut_multi` commands:

- `mir_alloc_raw_ptr_const_multi : Int -> MIRType -> MIRSetup MIRValue`
- `mir_alloc_raw_ptr_mut_multi : Int -> MIRType -> MIRSetup MIRValue`

The `Int` argument specifies how many values of the given type there are (*not* the size in bytes). If `mir_alloc_raw_ptr_{const,mut}_multi n` is used in the pre-state section of a specification (before `mir_execute_func`), it will create an allocation of `n` values, with the pointer pointing to the first value in that allocation. However, if used in the post-state section (after `mir_execute_func`), the raw pointer `MIRValue` is able to be matched against a raw pointer into a larger allocation produced by the function. The only requirement is that the pointer points to a contiguous sequence of `n` values within some allocation; the allocation is allowed to contain more values before or after those `n` values.

11.10 Specifying Heap Values

Pointers returned by `llvm_alloc`, `jvm_alloc_{array,object}`, or `mir_alloc{,_mut,_ptr_const,_ptr_mut}` don't initially point to anything. So if you pass such a pointer directly into a function that tried to dereference it, symbolic execution will fail with a message about an invalid load. For some functions, such as those that are intended to initialize data structures (writing to the memory pointed to, but never reading from it), this sort of uninitialized memory is appropriate. In most cases, however, it's more useful to state that a pointer points to some specific (usually symbolic) value, which you can do with the *points-to* family of commands.

11.10.1 LLVM heap values

LLVM verification primarily uses the `llvm_points_to` command:

- `llvm_points_to : LLVMValue -> LLVMValue -> LLVMSetup ()` takes two `LLVMValue` arguments, the first of which must be a pointer, and states that the memory specified by that pointer should contain the value given in the second argument (which may be any type of `LLVMValue`).

When used in the final state, `llvm_points_to` specifies that the given pointer *should* point to the given value when the function finishes.

Occasionally, because C programs frequently reinterpret memory of one type as another through casts, it can be useful to specify that a pointer points to a value that does not agree with its static type.

- `llvm_points_to_untyped : LLVMValue -> LLVMValue -> LLVMSetup ()` works like `llvm_points_to` but omits type checking. Rather than omitting type checking across the board, we introduced this additional function to make it clear when a type reinterpretation is intentional. As an alternative, one may instead use `llvm_cast_pointer` to line up the static types.

11.10.2 JVM heap values

JVM verification has two categories of commands for specifying heap values. One category consists of the `jvm_*_is` commands, which allow users to directly specify what value a heap object points to. There are specific commands for each type of JVM heap object:

- `jvm_array_is : JVMValue -> Term -> JVMSetup ()` declares that an array (the first argument) contains a sequence of values (the second argument).
- `jvm_elem_is : JVMValue -> Int -> JVMValue -> JVMSetup ()` declares that an array (the first argument) has an element at the given index (the second argument) containing the given value (the third argument).
- `jvm_field_is : JVMValue -> String -> JVMValue -> JVMSetup ()` declares that an object (the first argument) has a field (the second argument) containing the given value (the third argument).
- `jvm_static_field_is : String -> JVMValue -> JVMSetup ()` declares that a named static field (the first argument) contains the given value (the second argument). By default, the field name is assumed to belong to the same class as the method being specified. Static fields belonging to other classes can be selected using the `<classname>.<fieldname>` syntax in the first argument.

Another category consists of the `jvm_modifies_*` commands. Like the `jvm_*_is` commands, these specify that a JVM heap object points to valid memory, but unlike the `jvm_*_is` commands, they leave the exact value being pointed to as unspecified. These are useful for writing partial specifications for methods that modify some heap value, but without saying anything specific about the new value.

- `jvm_modifies_array : JVMValue -> JVMSetup ()`
- `jvm_modifies_elem : JVMValue -> Int -> JVMSetup ()`
- `jvm_modifies_field : JVMValue -> String -> JVMSetup ()`
- `jvm_modifies_static_field : String -> JVMSetup ()`

11.10.3 MIR heap values

MIR verification primarily uses the `mir_points_to` command:

- `mir_points_to : MIRValue -> MIRValue -> MIRSetup ()` takes two `MIRValue` arguments, the first of which must be a reference or raw pointer, and states that the memory specified by that reference or raw pointer should contain the value given in the second argument (which may be any type of `MIRValue`).

As a convenience, SAW also provides:

- `mir_ref_of : MIRValue -> MIRSetup MIRValue`
- `mir_ref_of_mut : MIRValue -> MIRSetup MIRValue`

which combine `mir_alloc`/`mir_alloc_mut` and `mir_points_to` into a single operation.

Some low-level Rust code involves casting raw pointers, resulting in raw pointers which point to values of a different type than what the raw pointer's static type claims. This can be modeled in SAW using the `mir_cast_raw_ptr` command:

- `mir_cast_raw_ptr : MIRValue -> MIRType -> MIRType` takes a raw pointer and a type, and returns a raw pointer to the same memory location and with the same mutability as the given pointer, but with the given type as the static pointee type instead.

Unlike in the LLVM backend, this does *not* allow for reinterpretation of memory. If a raw pointer points to an allocation that is actually of type `T`, the pointer can be cast and passed around and stored as a pointer to another type, but it must be casted back to `*T` when it is actually dereferenced. Accordingly, SAW enforces that `mir_points_to` can only be used on a non-casted pointer, so that the value in the second argument matches the type passed to the `mir_alloc_raw_ptr` that created the raw pointer in the first argument. `mir_cast_raw_ptr` can be used, though, whenever some Rust signature is expecting a pointer whose static pointee type does not match its "true" type at runtime.

For raw pointers to contiguous sequences of multiple values, created by `mir_alloc_raw_ptr_const_multi` and `mir_alloc_raw_ptr_mut_multi`, the `mir_points_to_multi` command can be used to specify the multiple values.

- `mir_points_to_multi : MIRValue -> MIRValue -> MIRSetup ()`

The second argument must have a MIR array type, and it specifies the sequence of pointed-to values as a MIR array. Specifically, if the first argument is a raw pointer to a contiguous sequence of `n` values of type `ty`, the second argument must have the MIR type `mir_array m ty` where `m <= n`. Note that the second argument need not be constructed with `mir_array_value`; it can also be derived from a fresh variable or a Cryptol sequence expression. Also note that the pointed-to values are not (necessarily) the contents of an array in the actual MIR semantics; their corresponding `MIRValues` are just represented as an array in SAWScript specs, for ease of conversion from Cryptol sequences.

11.11 Working with Compound Types

The commands mentioned so far give us no way to specify the values of compound types (arrays or `structs`). Compound values can be dealt with either piecewise or in their entirety.

- `llvm_elem : LLVMValue -> Int -> LLVMValue` yields a pointer to an internal element of a compound value. For arrays, the `Int` parameter is the array index. For `struct` values, it is the field index.
- `llvm_field : LLVMValue -> String -> LLVMValue` yields a pointer to a particular named `struct` field, if debugging information is available in the bitcode.

Either of these functions can be used with `llvm_points_to` to specify the value of a particular array element or `struct` field. Sometimes, however, it is more convenient to specify all array elements or field values at once. The `llvm_array_value` and `llvm_struct_value` functions construct compound values from lists of element values.

- `llvm_array_value : [LLVMValue] -> LLVMValue`
- `llvm_struct_value : [LLVMValue] -> LLVMValue`

To specify an array or struct in which each element or field is symbolic, it would be possible, but tedious, to use a large combination of `llvm_fresh_var` and `llvm_elem` or `llvm_field` commands.

However, the following function can simplify the common case where you want every element or field to have a fresh value.

- `llvm_fresh_expanded_val : LLVMType -> LLVMSetup LLVMValue`

The `llvm_struct_value` function normally creates a `struct` whose layout obeys the alignment rules of the platform specified in the LLVM file being analyzed. Structs in LLVM can explicitly be "packed", however, so that every field immediately follows the previous in memory. The following command will create values of such types:

- `llvm_packed_struct_value : [LLVMValue] -> LLVMValue`

C programs will sometimes make use of pointer casting to implement various kinds of polymorphic behaviors, either via direct pointer casts, or by using `union` types to codify the pattern. To reason about such cases, the following operation is useful.

- `llvm_cast_pointer : LLVMValue -> LLVMType -> LLVMValue`

This function function casts the type of the input value (which must be a pointer) so that it points to values of the given type. This mainly affects the results of subsequent `llvm_field` and `llvm_elem` calls, and any eventual `points_to` statements that the resulting pointer flows into. This is especially useful for dealing with C `union` types, as the type information provided by LLVM is imprecise in these cases.

We can automate the process of applying pointer casts if we have debug information avaliable:

- `llvm_union : LLVMValue -> String -> LLVMValue`

Given a pointer setup value, this attempts to select the named union branch and cast the type of the pointer. For this to work, debug symbols must be included; moreover, the process of correlating LLVM type information with information contained in debug symbols is a bit heuristic. If `llvm_union` cannot figure out how to cast a pointer, one can fall back on the more manual `llvm_cast_pointer` instead.

In the Java verification implementation, the following functions can be used to state the equivalent of a combination of `llvm_points_to` and either `llvm_elem` or `llvm_field`.

- `jvm_elem_is : JVMValue -> Int -> JVMValue -> JVMSetup ()` specifies the value of an array element.
- `jvm_field_is : JVMValue -> String -> JVMValue -> JVMSetup ()` specifies the name of an object field.

In the MIR verification implementation, the following functions construct compound values:

- `mir_array_value : MIRType -> [MIRValue] -> MIRValue` constructs an array of the given type whose elements consist of the given values. Supplying the element type is necessary to support length-0 arrays.
- `mir_enum_value : MIRAdt -> String -> [MIRValue] -> MIRValue` constructs an enum using a particular enum variant. The `MIRAdt` arguments determines what enum type

to create, the `String` value determines the name of the variant to use, and the `[MIRValue]` list are the values to use as elements in the variant.

See the "Finding MIR algebraic data types" section (as well as the "Enums" subsection) for more information on how to compute a `MIRAdt` value to pass to `mir_enum_value`.

- `mir_slice_value : MIRValue -> MIRValue`: see the "MIR slices" section below.
- `mir_slice_range_value : MIRValue -> Int -> Int -> MIRValue`: see the "MIR slices" section below.
- `mir_str_slice_value : MIRValue -> MIRValue`: see the "MIR slices" section below.
- `mir_str_slice_range_value : MIRValue -> Int -> Int -> MIRValue`: see the "MIR slices" section below.
- `mir_struct_value : MIRAdt -> [MIRValue] -> MIRValue` construct a struct with the given list of values as elements. The `MIRAdt` argument determines what struct type to create.

  See the "Finding MIR algebraic data types" section for more information on how to compute a `MIRAdt` value to pass to `mir_struct_value`.
- `mir_tuple_value : [MIRValue] -> MIRValue` construct a tuple with the given list of values as elements.

To specify a compound value in which each element or field is symbolic, it would be possible, but tedious, to use a large number of `mir_fresh_var` invocations in conjunction with the commands above. However, the following function can simplify the common case where you want every element or field to have a fresh value:

- `mir_fresh_expanded_value : String -> MIRType -> MIRSetup MIRValue`

The `String` argument denotes a prefix to use when generating the names of fresh symbolic variables. The `MIRType` can be any type, with the exception of reference types (or compound types that contain references as elements or fields), which are not currently supported.

The following functions extract components of compound MIR values:

- `mir_elem_value : MIRValue -> Int -> MIRValue` takes an array value and an index, and returns the value in the array at that index.
- `mir_elem_ref : MIRValue -> Int -> MIRValue` takes a reference (or raw pointer) to an array, and an index, and returns a reference (resp. raw pointer) to the element in the array at that index.

Note that unlike `llvm_elem`, `mir_elem_ref` cannot be used to specify the value of a specific index of an array reference without the whole array reference already being initialized.

11.11.1 MIR slices

Slices are a unique form of compound type that is currently only used during MIR verification. Unlike other forms of compound values, such as arrays, it is not possible to directly construct a slice. Instead, one must take a slice of an existing reference value that points to the thing being sliced.

SAW currently supports taking slices of arrays and strings.

Array slices

The following commands are used to construct slices of arrays:

- `mir_slice_value : MIRValue -> MIRValue`: the SAWScript expression `mir_slice_value base` is equivalent to the Rust expression `&base[..]`, i.e., a slice of the entirety of `base`. `base` must be a reference to an array value (`&[T; N]` or `&mut [T; N]`), not an array itself. The type of `mir_slice_value base` will be `&[T]` (if `base` is an immutable reference) or `&mut [T]` (if `base` is a mutable reference).
- `mir_slice_range_value : MIRValue -> Int -> Int -> MIRValue`: the SAWScript expression `mir_slice_range_value base start end` is equivalent to the Rust expression `&base[start..end]`, i.e., a slice over a part of `base` which ranges from `start` to `end`. `base` must be a reference to an array value (`&[T; N]` or `&mut [T; N]`), not an array itself. The type of `mir_slice_value base` will be `&[T]` (if `base` is an immutable reference) or `&mut [T]` (if `base` is a mutable reference).

  `start` and `end` are assumed to be zero-indexed. `start` must not exceed `end`, and `end` must not exceed the length of the array that `base` points to.

As an example of how to use these functions, consider this Rust function, which accepts an arbitrary slice as an argument:

```
pub fn f(s: &[u32]) -> u32 {
    s[0] + s[1]
}
```

We can write a specification that passes a slice to the array `[1, 2, 3, 4, 5]` as an argument to `f`:

```
let f_spec_1 = do {
  a <- mir_alloc (mir_array 5 mir_u32);
  mir_points_to a (mir_term {{ [1, 2, 3, 4, 5] : [5][32] }});

  mir_execute_func [mir_slice_value a];

  mir_return (mir_term {{ 3 : [32] }});
};
```

Alternatively, we can write a specification that passes a part of this array over the range `[1..3]`, i.e., ranging from second element to the fourth. Because this is a half-open range, the resulting slice has

length 2:

```
let f_spec_2 = do {
  a <- mir_alloc (mir_array 5 mir_u32);
  mir_points_to a (mir_term {{ [1, 2, 3, 4, 5] : [5][32] }});

  mir_execute_func [mir_slice_range_value a 1 3];

  mir_return (mir_term {{ 5 : [32] }});
};
```

Note that we are passing *references* of arrays to `mir_slice_value` and `mir_slice_range_value`. It would be an error to pass a bare array to these functions, so the following specification would be invalid:

```
let f_fail_spec_ = do {
  let arr = mir_term {{ [1, 2, 3, 4, 5] : [5][32] }};

  mir_execute_func [mir_slice_value arr]; // BAD: `arr` is not a␣
↪reference

  mir_return (mir_term {{ 3 : [32] }});
};
```

Note that The `mir_slice_range_value` function must accept bare `Int` arguments to specify the lower and upper bounds of the range. A consequence of this design is that it is not possible to create a slice with a symbolic length. If this limitation prevents you from using SAW, please file an issue on GitHub (https://github.com/GaloisInc/saw-script/issues).

String slices

In addition to slices of arrays (i.e., of type `&[T]`), SAW also supports slices of strings (i.e., of type `&str`) through the following commands:

- `mir_str_slice_value : MIRValue -> MIRValue`: the SAWScript expression `mir_str_slice_value base` is equivalent to the Rust expression `&base[..]`, i.e., a slice of the entirety of `base`. `base` must be a reference to an array of bytes (`&[u8; N]` or `&mut [u8; N]`), not an array itself. The type of `mir_str_slice_value base` will be `&str` (if `base` is an immutable reference) or `&mut str` (if `base` is a mutable reference).
- `mir_str_slice_range_value : MIRValue -> Int -> Int -> MIRValue`: the SAWScript expression `mir_slice_range_value base start end` is equivalent to the Rust expression `&base[start..end]`, i.e., a slice over a part of `base` which ranges from `start` to `end`. `base` must be a reference to an array of bytes (`&[u8; N]` or `&mut [u8; N]`), not an array itself. The type of `mir_slice_value base` will be `&str` (if `base` is an immutable reference) or `&mut str` (if `base` is a mutable reference).

`start` and `end` are assumed to be zero-indexed. `start` must not exceed `end`, and `end` must not exceed the length of the array that `base` points to.

One unusual requirement about `mir_str_slice_value` and `mir_str_slice_range_value` is that they require the argument to be of type `&[u8; N]`, i.e., a reference to an array of bytes. This is an artifact of the way that strings are encoded in Cryptol. The following Cryptol expressions:

- `"A"`
- `"123"`
- `"Hello World"`

Have the following types:

- `[1][8]`
- `[3][8]`
- `[11][8]`

This is because Cryptol strings are syntactic shorthand for sequences of bytes. The following Cryptol expressions are wholly equivalent:

- `[0x41]`
- `[0x31, 0x32, 0x33]`
- `[0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0x64]`

These represent the strings in the extended ASCII character encoding. The Cryptol sequence type `[N][8]` is equivalent to the Rust type `[u8; N]`, so the requirement to have something of type `&[u8; N]` as an argument reflects this design choice.

Note that `mir_str_slice_value <u8_array_ref>` is *not* the same thing as `mir_slice_value <u8_array_ref>`, as the two commands represent different types of Rust values. While both commands take a `<u8_array_ref>` as an argument, `mir_str_slice_value` will return a value of Rust type `&str` (or `&mut str`), whereas `mir_slice_value` will return a value of Rust type `&[u8]` (or `&mut [u8]`). These Rust types are checked when you pass these values as arguments to Rust functions (using `mir_execute_func`) or when you return these values (using `mir_return`), and it is an error to supply a `&str` value in a place where a `&[u8]` value is expected (and vice versa).

As an example of how to write specifications involving string slices, consider this Rust function:

```
pub fn my_len(s: &str) -> usize {
    s.len()
}
```

We can use `mir_str_slice_value` to write a specification for `my_len` when it is given the string `"hello"` as an argument:

```
let my_len_spec = do {
  s <- mir_alloc (mir_array 5 mir_u8);
  mir_points_to s (mir_term {{ "hello" }});

  mir_execute_func [mir_str_slice_value s];

  mir_return (mir_term {{ 5 : [64] }});
};
```

Currently, Cryptol only supports characters that can be encoded in a single byte. As a result, it is not currently possible to take slices of strings with certain characters. For example, the string `"roșu"` cannot be used as a Cryptol expression, as the character `'ș'` would require 10 bits to represent instead of 8. The alternative is to use UTF-8 to encode such characters. For instance, the UTF-8 encoding of the string `"roșu"` is `"ro\200\153u"`, where `"\200\153"` is a sequence of two bytes that represents the `'ș'` character.

SAW makes no attempt to ensure that string slices over a particular range aligns with UTF-8 character boundaries. For example, the following Rust code would panic:

```
    let rosu: &str = "roșu";
    let s: &str = &rosu[0..3];
    println!("{:?}", s);
```

```
thread 'main' panicked at 'byte index 3 is not a char boundary; it is␣
↪inside 'ș' (bytes 2..4) of `roșu`'
```

On the other hand, SAW will allow you define a slice of the form `mir_str_slice_range r 0 3`, where `r` is a reference to `"ro\200\153u"`. It is the responsibility of the SAW user to ensure that `mir_str_slice_range` indices align with character boundaries.

11.11.2 MIR `Vec`s

`Vec` (https://doc.rust-lang.org/std/vec/struct.Vec.html) is a commonly used data type in the Rust standard library. `Vec` values can be created from array values in MIR specifications with the following command:

- `mir_vec_of : String -> MIRType -> MIRValue -> MIRSetup MIRValue`

The `String` argument is used as a prefix for naming the internal symbolic variables created as part of the `Vec` struct (think of it just as a name you give to the `Vec` variable). The `MIRType` argument is the element type of the `Vec`. The `MIRValue` argument is the contents of the `Vec`, which must be a MIR array value whose element type matches the `MIRType` argument. Note that this could either be created with `mir_array_value` or obtained from a `Term` like a fresh variable or a Cryptol sequence expression.

`Vec` is just a regular struct and not a special language construct, so technically you could write

specifications for `Vec`s just using the primitive MIR specification commands (in fact, this is what `mir_vec_of` does internally). However, you would need to explicitly specify all the internal details and invariants of `Vec`, and that can get quite messy. Therefore, this command exists for convenience reasons.

11.11.3 Finding MIR algebraic data types

We collectively refer to MIR `struct`s and `enum`s together as *algebraic data types*, or ADTs for short. ADTs have identifiers to tell them apart, and a single ADT declaration can give rise to multiple identifiers depending on how the declaration is used. For example:

```
pub struct S<A, B> {
    pub x: A,
    pub y: B,
}

pub fn f() -> S<u8, u16> {
    S {
        x: 1,
        y: 2,
    }
}

pub fn g() -> S<u32, u64> {
    S {
        x: 3,
        y: 4,
    }
}
```

This program as a single `struct` declaration `S`, which is used in the functions `f` and `g`. Note that `S`'s declaration is *polymorphic*, as it uses type parameters, but the uses of `S` in `f` and `g` are *monomorphic*, as `S`'s type parameters are fully instantiated. Each unique, monomorphic instantiation of an ADT gives rise to its own identifier. In the example above, this might mean that the following identifiers are created when this code is compiled with `mir-json`:

- `S<u8, u16>` gives rise to `example/abcd123::S::_adt456`
- `S<u32, u64>` gives rise to `example/abcd123::S::_adt789`

The suffix `_adt<number>` is autogenerated by `mir-json` and is typically difficult for humans to guess. For this reason, we offer a command to look up an ADT more easily:

- `mir_find_adt : MIRModule -> String -> [MIRType] -> MIRAdt` consults the given `MIRModule` to find an algebraic data type (`MIRAdt`). It uses the given `String` as an identifier and the given MIRTypes as the types to instantiate the type parameters of the ADT. If such a `MIRAdt` cannot be found in the `MIRModule`, this will raise an error.

Note that the `String` argument to `mir_find_adt` does not need to include the `_adt<num>` suffix, as `mir_find_adt` will discover this for you. The `String` is expected to adhere to the identifier conventions described in the "Running a MIR-based verification" section. For instance, the following two lines will look up `S<u8, u16>` and `S<u32, u64>` from the example above as `MIRAdt`s:

```
m <- mir_load_module "example.linked-mir.json";

let s_8_16  = mir_find_adt m "example::S" [mir_u8,  mir_u16];
let s_32_64 = mir_find_adt m "example::S" [mir_u32, mir_u64];
```

See also the *const generics* section for more details on how to look up `MIRAdt`s that use const generics.

Note that there is also a command to look up ADTs by their full, *mangled* identifiers that include the `_adt<num>` suffix:

- `mir_find_mangled_adt : MIRModule -> String -> MIRAdt`

Note that unlike `mir_find_adt`, `mir_find_mangled_adt` lacks `[MirType]` arguments, as the type information is already encoded into the mangled identifier.

It is recommended to use `mir_find_adt` over `mir_find_mangled_adt` whenever possible, as mangled identifiers can change easily when recompiling Rust code. `mir_find_mangled_adt` is generally only needed to work around limitations in what `mir_find_adt` can look up.

The `mir_adt` command (for constructing a struct type), `mir_struct_value` (for constructing a struct value), and `mir_enum_value` (for constructing an enum value) commands in turn take a `MIRAdt` as an argument.

Enums

In addition to taking a `MIRAdt` as an argument, `mir_enum_value` also takes a `String` representing the name of the variant to construct. The variant name should be a short name such as `"None"` or `"Some"`, and not a full identifier such as `"core::option::Option::None"` or `"core::option::Option::Some"`. This is because the `MIRAdt` already contains the full identifiers for all of an enum's variants, so SAW will use this information to look up a variant's identifier from a short name. Here is an example of using `mir_enum_value` in practice:

```
pub fn n() -> Option<u32> {
    None
}

pub fn s(x: u32) -> Option<u32> {
    Some(x)
}
```

```
m <- mir_load_module "example.linked-mir.json";
```

(continues on next page)

(continued from previous page)

```
let option_u32 = mir_find_adt m "core::option::Option" [mir_u32];

let n_spec = do {
  mir_execute_func [];

  mir_return (mir_enum_value option_u32 "None" []);
};

let s_spec = do {
  x <- mir_fresh_var "x" mir_u32;

  mir_execute_func [mir_term x];

  mir_return (mir_enum_value option_u32 "Some" [mir_term x]);
};
```

Note that `mir_enum_value` can only be used to construct a specific variant. If you need to construct a symbolic enum value that can range over many potential variants, use `mir_fresh_expanded_value` instead.

Lifetimes

Rust ADTs can have both type parameters as well as *lifetime* parameters. The following Rust code declares a lifetime parameter `'a` on the struct `S`, as well on the function `f` that computes an `S` value:

```
pub struct S<'a> {
    pub x: &'a u32,
}

pub fn f<'a>(y: &'a u32) -> S<'a> {
    S { x: y }
}
```

When `mir-json` compiles a piece of Rust code that contains lifetime parameters, it will instantiate all of the lifetime parameters with a placeholder MIR type that is simply called `lifetime`. This is important to keep in mind when looking up ADTs with `mir_find_adt`, as you will also need to indicate to SAW that the lifetime parameter is instantiated with `lifetime`. In order to do so, use `mir_lifetime`. For example, here is how to look up `S` with `'a` instantiated to `lifetime`:

```
s_adt = mir_find_adt m "example::S" [mir_lifetime]
```

Note that this part of SAW's design is subject to change in the future. Ideally, users would not have to care about lifetimes at all at the MIR level; see this issue (https://github.com/GaloisInc/mir-json/issues/58) for further discussion on this point. If that issue is fixed, then we will likely remove

`mir_lifetime`, as it will no longer be necessary.

Const generics

Rust ADTs can have *const generic* parameters that allow the ADT to be generic over constant values. For instance, the following Rust code declares a const generic parameter `N` on the struct `S`, as well as on the functions `f` and `g` that compute `S` values:

```
pub struct S<const N: usize> {
    pub x: [u32; N]
}

pub fn f(y: [u32; 1]) -> S<1> {
    S { x: y }
}

pub fn g(y: [u32; 2]) -> S<2> {
    S { x: y }
}
```

Like with other forms of Rust generics, instantiating `S` with different constants will give rise to different identifiers in the compiled MIR code. SAW provides a `mir_const` function for specifying the values of constants used to instantiate const generic parameters:

- `mir_const : MIRType -> Term -> MIRType`

For instance, if order to look up `S<1>`, use `mir_const` in conjunction with `mir_find_adt` like so:

```
s_adt = mir_find_adt m "example::S" [mir_const mir_usize {{ 1 : [64] }}]
```

Unlike other forms of `MIRType`s, the type returned by `mir_const` is not a type that you can create values with. For instance, calling `mir_alloc` or `mir_fresh_var` at a type returned by `mir_const` will raise an error. `mir_const` is only useful for looking up ADTs via `mir_find_adt`.

At present, `mir_const` only supports looking up constant values with the types listed here (https://doc.rust-lang.org/1.86.0/reference/items/generics.html#r-items.generics.const.allowed-types) in the Rust Reference. Specifically, the `MIRType` argument must be one of the following, subject to the following restrictions:

- A primitive integer type, i.e., `mir_u{8,16,32,64,128,size}` or `mir_i{8,16,32,64,128,size}`. The `Term` argument must be a bitvector of the corresponding size. For instance, if the `MIRType` is `mir_u8`, then the `Term` must be a bitvector of type `[8]`.
- `mir_bool`. The `Term` argument must be of type `Bit`.
- `mir_char`. The `Term` argument must be of type `[32]`.

11.11.4 Bitfields

SAW has experimental support for specifying `struct`s with bitfields, such as in the following example:

```
struct s {
  uint8_t x:1;
  uint8_t y:1;
};
```

Normally, a `struct` with two `uint8_t` fields would have an overall size of two bytes. However, because the `x` and `y` fields are declared with bitfield syntax, they are instead packed together into a single byte.

Because bitfields have somewhat unusual memory representations in LLVM, some special care is required to write SAW specifications involving bitfields. For this reason, there is a dedicated `llvm_points_to_bitfield` function for this purpose:

- `llvm_points_to_bitfield : LLVMValue -> String -> LLVMValue -> LLVMSetup ()`

The type of `llvm_points_to_bitfield` is similar that of `llvm_points_to`, except that it takes the name of a field within a bitfield as an additional argument. For example, here is how to assert that the `y` field in the `struct` example above should be `0`:

```
ss <- llvm_alloc (llvm_alias "struct.s");
llvm_points_to_bitfield ss "y" (llvm_term {{ 0 : [1] }});
```

Note that the type of the right-hand side value (`0`, in this example) must be a bitvector whose length is equal to the size of the field within the bitfield. In this example, the `y` field was declared as `y:1`, so `y`'s value must be of type `[1]`.

Note that the following specification is *not* equivalent to the one above:

```
ss <- llvm_alloc (llvm_alias "struct.s");
llvm_points_to (llvm_field ss "y") (llvm_term {{ 0 : [1] }});
```

`llvm_points_to` works quite differently from `llvm_points_to_bitfield` under the hood, so using `llvm_points_to` on bitfields will almost certainly not work as expected.

In order to use `llvm_points_to_bitfield`, one must also use the `enable_lax_loads_and_stores` command:

- `enable_lax_loads_and_stores: TopLevel ()`

Both `llvm_points_to_bitfield` and `enable_lax_loads_and_stores` are experimental commands, so these also require using `enable_experimental` before they can be used.

The `enable_lax_loads_and_stores` command relaxes some of SAW's assumptions about uninitialized memory, which is necessary to make `llvm_points_to_bitfield` work under the hood.

For example, reading from uninitialized memory normally results in an error in SAW, but with `enable_lax_loads_and_stores`, such a read will instead return a symbolic value. At present, `enable_lax_loads_and_stores` only works with What4-based tactics (e.g., `w4_unint_z3`); using it with SBV-based tactics (e.g., `sbv_unint_z3`) will result in an error.

Note that SAW relies on LLVM debug metadata in order to determine which struct fields reside within a bitfield. As a result, you must pass `-g` to `clang` when compiling code involving bitfields in order for SAW to be able to reason about them.

11.12 Global variables

SAW supports verifying LLVM and MIR specifications involving global variables.

11.12.1 LLVM global variables

Mutable global variables that are accessed in a function must first be allocated by calling `llvm_alloc_global` on the name of the global.

- `llvm_alloc_global : String -> LLVMSetup ()`

This ensures that all global variables that might influence the function are accounted for explicitly in the specification: if `llvm_alloc_global` is used in the precondition, there must be a corresponding `llvm_points_to` in the postcondition describing the new state of that global. Otherwise, a specification might not fully capture the behavior of the function, potentially leading to unsoundness in the presence of compositional verification. (For more details on this point, see the *Compositional Verification and Mutable Global Variables* section.)

Immutable (i.e. `const`) global variables are allocated implicitly, and do not require a call to `llvm_alloc_global`.

Pointers to global variables or functions can be accessed with `llvm_global`:

- `llvm_global : String -> LLVMValue`

Like the pointers returned by `llvm_alloc`, however, these aren't initialized at the beginning of symbolic – setting global variables may be unsound in the presence of *compositional verification*.

To understand the issues surrounding global variables, consider the following C code:

```
int x = 0;

int f(int y) {
  x = x + 1;
  return x + y;
}

int g(int z) {
```

(continues on next page)

(continued from previous page)

```
  x = x + 2;
  return x + z;
}
```

One might initially write the following specifications for `f` and `g`:

```
m <- llvm_load_module "./test.bc";

f_spec <- llvm_verify m "f" [] true (do {
    y <- llvm_fresh_var "y" (llvm_int 32);
    llvm_execute_func [llvm_term y];
    llvm_return (llvm_term {{ 1 + y : [32] }});
}) abc;

g_spec <- llvm_llvm_verify m "g" [] true (do {
    z <- llvm_fresh_var "z" (llvm_int 32);
    llvm_execute_func [llvm_term z];
    llvm_return (llvm_term {{ 2 + z : [32] }});
}) abc;
```

If globals were always initialized at the beginning of verification, both of these specs would be provable. However, the results wouldn't truly be compositional. For instance, it's not the case that `f(g(z)) == z + 3` for all `z`, because both `f` and `g` modify the global variable `x` in a way that crosses function boundaries.

To deal with this, we can use the following function:

- `llvm_global_initializer : String -> LLVMValue` returns the value of the constant global initializer for the named global variable.

Given this function, the specifications for `f` and `g` can make this reliance on the initial value of `x` explicit:

```
m <- llvm_load_module "./test.bc";


let init_global name = do {
  llvm_alloc_global name;
  llvm_points_to (llvm_global name)
                 (llvm_global_initializer name);
};

f_spec <- llvm_verify m "f" [] true (do {
    y <- llvm_fresh_var "y" (llvm_int 32);
    init_global "x";
```

(continues on next page)

(continued from previous page)

```
    llvm_precond {{ y < 2^^31 - 1 }};
    llvm_execute_func [llvm_term y];
    llvm_return (llvm_term {{ 1 + y : [32] }});
}) abc;
```

which initializes `x` to whatever it is initialized to in the C code at the beginning of verification. This specification is now safe for compositional verification: SAW won't use the specification `f_spec` unless it can determine that `x` still has its initial value at the point of a call to `f`. This specification also constrains `y` to prevent signed integer overflow resulting from the `x + y` expression in `f`, which is undefined behavior in C.

11.12.2 MIR static items

Rust's static items are the MIR version of global variables. A reference to a static item can be accessed with the `mir_static` function. This function takes a `String` representing a static item's identifier, and this identifier is expected to adhere to the naming conventions outlined in the "Running a MIR-based verification" section:

- `mir_static : String -> MIRValue`

References to static values can be initialized with the `mir_points_to` command, just like with other forms of references. Immutable static items (e.g., `static X: u8 = 42`) are initialized implicitly in every SAW specification, so there is no need for users to do so manually. Mutable static items (e.g., `static mut Y: u8 = 27`), on the other hand, are *not* initialized implicitly, and users must explicitly pick a value to initialize them with.

The `mir_static_initializer` function can be used to access the initial value of a static item in a MIR program. Like with `mir_static`, the `String` supplied as an argument must be a valid identifier:

- `mir_static_initializer : String -> MIRValue`.

As an example of how to use these functions, here is a Rust program involving static items:

```
// statics.rs
static     S1: u8 = 1;
static mut S2: u8 = 2;

pub fn f() -> u8 {
    // Reading a mutable static item requires an `unsafe` block due to
    // concurrency-related concerns. We are only concerned about the␣
↪behavior
    // of this program in a single-threaded context, so this is fine.
    let s2 = unsafe { S2 };
    S1 + s2
}
```

We can write a specification for `f` like so:

```
// statics.saw

let f_spec = do {
  mir_points_to (mir_static "statics::S2")
                (mir_static_initializer "statics::S2");
  // Note that we do not initialize S1, as immutable static items are␣
↪implicitly
  // initialized in every specification.

  mir_execute_func [];

  mir_return (mir_term {{ 3 : [8] }});
};

m <- mir_load_module "statics.linked-mir.json";

mir_verify m "statics::f" [] false f_spec z3;
```

In order to use a specification involving mutable static items for compositional verification, it is required to specify the value of all mutable static items using the `mir_points_to` command in the specification's postconditions. For more details on this point, see the *Compositional Verification and Mutable Global Variables* section.

11.13 Preconditions and Postconditions

Sometimes a function is only well-defined under certain conditions, or sometimes you may be interested in certain initial conditions that give rise to specific final conditions. For these cases, you can specify an arbitrary predicate as a precondition or post-condition, using any values in scope at the time.

- `llvm_precond : Term -> LLVMSetup ()`
- `llvm_postcond : Term -> LLVMSetup ()`
- `llvm_assert : Term -> LLVMSetup ()`
- `jvm_precond : Term -> JVMSetup ()`
- `jvm_postcond : Term -> JVMSetup ()`
- `jvm_assert : Term -> JVMSetup ()`
- `mir_precond : Term -> MIRSetup ()`
- `mir_postcond : Term -> MIRSetup ()`
- `mir_assert : Term -> MIRSetup ()`

These commands take `Term` arguments, and therefore cannot describe the values of pointers. The "assert" variants will work in either pre- or post-conditions, and are useful when defining helper functions that, e.g., provide datastructure invariants that make sense in both phases. The `{llvm, jvm,mir}_equal` commands state that two values should be equal, and can be used in either the initial or the final state.

- `llvm_equal : LLVMValue -> LLVMValue -> LLVMSetup ()`
- `jvm_equal : JVMValue -> JVMValue -> JVMSetup ()`
- `mir_equal : MIRValue -> MIRValue -> MIRSetup ()`

The use of `{llvm,jvm,mir}_equal` can also sometimes lead to more efficient symbolic execution when the predicate of interest is an equality.

11.14 Assuming specifications

Normally, a `MethodSpec` is the result of both simulation and proof of the target code. However, in some cases, it can be useful to use a `MethodSpec` to specify some code that either doesn't exist or is hard to prove. The previously-mentioned *`assume_unsat` tactic* omits proof but does not prevent simulation of the function. To skip simulation altogether, one can use one of the following commands:

- `llvm_unsafe_assume_spec : LLVMModule -> String -> LLVMSetup () -> TopLevel CrucibleMethodSpec`
- `jvm_unsafe_assume_spec : JavaClass -> String -> JVMSetup () -> TopLevel JVMMethodSpec`
- `mir_unsafe_assume_spec : MIRModule -> String -> MIRSetup () -> TopLevel MIRSpec`

11.15 A Heap-Based Example

To tie all of the command descriptions from the previous sections together, consider the case of verifying the correctness of a C program that computes the dot product of two vectors, where the length and value of each vector are encapsulated together in a `struct`.

The dot product can be concisely specified in Cryptol as follows:

```
dotprod : {n, a} (fin n, fin a) => [n][a] -> [n][a] -> [a]
dotprod xs ys = sum (zip (*) xs ys)
```

To implement this in C, let's first consider the type of vectors:

```
typedef struct {
    uint32_t *elts;
```

(continues on next page)

(continued from previous page)

```
    uint32_t size;
} vec_t;
```

This struct contains a pointer to an array of 32-bit elements, and a 32-bit value indicating how many elements that array has.

We can compute the dot product of two of these vectors with the following C code (which uses the size of the shorter vector if they differ in size).

```
uint32_t dotprod_struct(vec_t *x, vec_t *y) {
    uint32_t size = MIN(x->size, y->size);
    uint32_t res = 0;
    for(size_t i = 0; i < size; i++) {
        res += x->elts[i] * y->elts[i];
    }
    return res;
}
```

The entirety of this implementation can be found in the `examples/llvm/dotprod_struct.c` file in the `saw-script` repository.

To verify this program in SAW, it will be convenient to define a couple of utility functions (which are generally useful for many heap-manipulating programs). First, combining allocation and initialization to a specific value can make many scripts more concise:

```
let alloc_init ty v = do {
    p <- llvm_alloc ty;
    llvm_points_to p v;
    return p;
};
```

This creates a pointer `p` pointing to enough space to store type `ty`, and then indicates that the pointer points to value `v` (which should be of that same type).

A common case for allocation and initialization together is when the initial value should be entirely symbolic.

```
let ptr_to_fresh n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init ty (llvm_term x);
    return (x, p);
};
```

This function returns the pointer just allocated along with the fresh symbolic value it points to.

Given these two utility functions, the `dotprod_struct` function can be specified as follows:

```
let dotprod_spec n = do {
    let nt = llvm_term {{ `n : [32] }};
    (xs, xsp) <- ptr_to_fresh "xs" (llvm_array n (llvm_int 32));
    (ys, ysp) <- ptr_to_fresh "ys" (llvm_array n (llvm_int 32));
    let xval = llvm_struct_value [ xsp, nt ];
    let yval = llvm_struct_value [ ysp, nt ];
    xp <- alloc_init (llvm_alias "struct.vec_t") xval;
    yp <- alloc_init (llvm_alias "struct.vec_t") yval;
    llvm_execute_func [xp, yp];
    llvm_return (llvm_term {{ dotprod xs ys }});
};
```

Any instantiation of this specification is for a specific vector length `n`, and assumes that both input vectors have that length. That length `n` automatically becomes a type variable in the subsequent Cryptol expressions, and the backtick operator is used to reify that type as a bit vector of length 32.

The entire script can be found in the `dotprod_struct-crucible.saw` file alongside `dotprod_struct.c`.

Running this script results in the following:

```
Loading file "dotprod_struct.saw"
Proof succeeded! dotprod_struct
Registering override for `dotprod_struct`
  variant `dotprod_struct`
Symbolic simulation completed with side conditions.
Proof succeeded! dotprod_wrap
```

11.16 Using Ghost State

In some cases, information relevant to verification is not directly present in the concrete state of the program being verified. This can happen for at least two reasons:

- When providing specifications for external functions, for which source code is not present. The external code may read and write global state that is not directly accessible from the code being verified.
- When the abstract specification of the program naturally uses a different representation for some data than the concrete implementation in the code being verified does.

One solution to these problems is the use of *ghost* state. This can be thought of as additional global state that is visible only to the verifier. Ghost state with a given name can be declared at the top level with the following function:

- `declare_ghost_state : String -> TopLevel Ghost`

Ghost state variables do not initially have any particluar type, and can store data of any type. Given an existing ghost variable the following functions can be used to specify its value:

- `llvm_ghost_value : Ghost -> Term -> LLVMSetup ()`
- `jvm_ghost_value : Ghost -> Term -> JVMSetup ()`
- `mir_ghost_value : Ghost -> Term -> MIRSetup ()`

These can be used in either the pre state or the post state, to specify the value of ghost state either before or after the execution of the function, respectively.

11.17 An Extended Example

To tie together many of the concepts in this manual, we now present a non-trivial verification task in its entirety. All of the code for this example can be found in the `examples/salsa20` directory of the SAWScript repository (https://github.com/GaloisInc/saw-script).

11.17.1 Salsa20 Overview

Salsa20 is a stream cipher developed in 2005 by Daniel J. Bernstein, built on a pseudorandom function utilizing add-rotate-XOR (ARX) operations on 32-bit words[4]. Bernstein himself has provided several public domain implementations of the cipher, optimized for common machine architectures. For the mathematically inclined, his specification for the cipher can be found here (http://cr.yp.to/snuffle/spec.pdf).

The repository referenced above contains three implementations of the Salsa20 cipher: A reference Cryptol implementation (which we take as correct in this example), and two C implementations, one of which is from Bernstein himself. For this example, we focus on the second of these C implementations, which more closely matches the Cryptol implementation. Full verification of Bernstein's implementation is available in `examples/salsa20/djb`, for the interested. The code for this verification task can be found in the files named according to the pattern `examples/salsa20/(s|S)alsa20.*`.

11.17.2 Specifications

We now take on the actual verification task. This will be done in two stages: We first define some useful utility functions for constructing common patterns in the specifications for this type of program (i.e. one where the arguments to functions are modified in-place.) We then demonstrate how one might construct a specification for each of the functions in the Salsa20 implementation described above.

[4] https://en.wikipedia.org/wiki/Salsa20

Utility Functions

We first define the function `alloc_init : LLVMType -> Term -> LLVMSetup LLVMValue`.

`alloc_init ty v` returns a `LLVMValue` consisting of a pointer to memory allocated and initialized to a value `v` of type `ty`. `alloc_init_readonly` does the same, except the memory allocated cannot be written to.

```
import "Salsa20.cry";

let alloc_init ty v = do {
    p <- llvm_alloc ty;
    llvm_points_to p (llvm_term v);
    return p;
};

let alloc_init_readonly ty v = do {
    p <- llvm_alloc_readonly ty;
    llvm_points_to p (llvm_term v);
    return p;
};
```

We now define `ptr_to_fresh : String -> LLVMType -> LLVMSetup (Term, LLVM-Value)`.

`ptr_to_fresh n ty` returns a pair `(x, p)` consisting of a fresh symbolic variable `x` of type `ty` and a pointer `p` to it. `n` specifies the name that SAW should use when printing `x`. `ptr_to_fresh_readonly` does the same, but returns a pointer to space that cannot be written to.

```
let ptr_to_fresh n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init ty x;
    return (x, p);
};

let ptr_to_fresh_readonly n ty = do {
    x <- llvm_fresh_var n ty;
    p <- alloc_init_readonly ty x;
    return (x, p);
};
```

Finally, we define `oneptr_update_func : String -> LLVMType -> Term -> LLVMSetup ()`.

`oneptr_update_func n ty f` specifies the behavior of a function that takes a single pointer (with a printable name given by `n`) to memory containing a value of type `ty` and mutates the contents of

that memory. The specification asserts that the contents of this memory after execution are equal to the value given by the application of `f` to the value in that memory before execution.

```
let oneptr_update_func n ty f = do {
    (x, p) <- ptr_to_fresh n ty;
    llvm_execute_func [p];
    llvm_points_to p (llvm_term {{ f x }});
};
```

The `quarterround` operation

The C function we wish to verify has type `void s20_quarterround(uint32_t *y0, uint32_t *y1, uint32_t *y2, uint32_t *y3)`.

The function's specification generates four symbolic variables and pointers to them in the precondition/setup stage. The pointers are passed to the function during symbolic execution via `llvm_execute_func`. Finally, in the postcondition/return stage, the expected values are computed using the trusted Cryptol implementation and it is asserted that the pointers do in fact point to these expected values.

```
let quarterround_setup : LLVMSetup () = do {
    (y0, p0) <- ptr_to_fresh "y0" (llvm_int 32);
    (y1, p1) <- ptr_to_fresh "y1" (llvm_int 32);
    (y2, p2) <- ptr_to_fresh "y2" (llvm_int 32);
    (y3, p3) <- ptr_to_fresh "y3" (llvm_int 32);

    llvm_execute_func [p0, p1, p2, p3];

    let zs = {{ quarterround [y0,y1,y2,y3] }}; // from Salsa20.cry
    llvm_points_to p0 (llvm_term {{ zs@0 }});
    llvm_points_to p1 (llvm_term {{ zs@1 }});
    llvm_points_to p2 (llvm_term {{ zs@2 }});
    llvm_points_to p3 (llvm_term {{ zs@3 }});
};
```

Simple Updating Functions

The following functions can all have their specifications given by the utility function `oneptr_update_func` implemented above, so there isn't much to say about them.

```
let rowround_setup =
    oneptr_update_func "y" (llvm_array 16 (llvm_int 32)) {{ rowround }};

let columnround_setup =
    oneptr_update_func "x" (llvm_array 16 (llvm_int 32)) {{ columnround␣
```

(continues on next page)

(continued from previous page)

```
↪}};

let doubleround_setup =
    oneptr_update_func "x" (llvm_array 16 (llvm_int 32)) {{ doubleround␣
↪}};

let salsa20_setup =
    oneptr_update_func "seq" (llvm_array 64 (llvm_int 8)) {{ Salsa20 }};
```

32-Bit Key Expansion

The next function of substantial behavior that we wish to verify has the following prototype:

```
void s20_expand32( uint8_t *k
                 , uint8_t n[static 16]
                 , uint8_t keystream[static 64]
                 )
```

This function's specification follows a similar pattern to that of `s20_quarterround`, though for extra assurance we can make sure that the function does not write to the memory pointed to by `k` or `n` using the utility `ptr_to_fresh_readonly`, as this function should only modify `keystream`. Besides this, we see the call to the trusted Cryptol implementation specialized to `a=2`, which does 32-bit key expansion (since the Cryptol implementation can also specialize to `a=1` for 16-bit keys). This specification can easily be changed to work with 16-bit keys.

```
let salsa20_expansion_32 = do {
    (k, pk) <- ptr_to_fresh_readonly "k" (llvm_array 32 (llvm_int 8));
    (n, pn) <- ptr_to_fresh_readonly "n" (llvm_array 16 (llvm_int 8));

    pks <- llvm_alloc (llvm_array 64 (llvm_int 8));

    llvm_execute_func [pk, pn, pks];

    let rks = {{ Salsa20_expansion`{a=2}(k, n) }};
    llvm_points_to pks (llvm_term rks);
};
```

32-bit Key Encryption

Finally, we write a specification for the encryption function itself, which has type

```
enum s20_status_t s20_crypt32( uint8_t *key
                             , uint8_t nonce[static 8]
```

(continues on next page)

(continued from previous page)

```
                                , uint32_t si
                                , uint8_t *buf
                                , uint32_t buflen
                                )
```

As before, we can ensure this function does not modify the memory pointed to by `key` or `nonce`. We take `si`, the stream index, to be 0. The specification is parameterized on a number `n`, which corresponds to `buflen`. Finally, to deal with the fact that this function returns a status code, we simply specify that we expect a success (status code 0) as the return value in the postcondition stage of the specification.

```
let s20_encrypt32 n = do {
    (key, pkey) <- ptr_to_fresh_readonly "key" (llvm_array 32 (llvm_int␣
↪8));
    (v, pv) <- ptr_to_fresh_readonly "nonce" (llvm_array 8 (llvm_int␣
↪8));
    (m, pm) <- ptr_to_fresh "buf" (llvm_array n (llvm_int 8));

    llvm_execute_func [ pkey
                      , pv
                      , llvm_term {{ 0 : [32] }}
                      , pm
                      , llvm_term {{ `n : [32] }}
                      ];

    llvm_points_to pm (llvm_term {{ Salsa20_encrypt (key, v, m) }});
    llvm_return (llvm_term {{ 0 : [32] }});
};
```

11.17.3 Verifying Everything

Finally, we can verify all of the functions. Notice the use of compositional verification and that path satisfiability checking is enabled for those functions with loops not bounded by explicit constants. Notice that we prove the top-level function for several sizes; this is due to the limitation that SAW can only operate on finite programs (while Salsa20 can operate on any input size.)

```
let main : TopLevel () = do {
    m      <- llvm_load_module "salsa20.bc";
    qr     <- llvm_verify m "s20_quarterround" []      false␣
↪quarterround_setup   abc;
    rr     <- llvm_verify m "s20_rowround"     [qr]    false rowround_
↪setup       abc;
    cr     <- llvm_verify m "s20_columnround"  [qr]    false␣
```

(continues on next page)

(continued from previous page)

```
↪columnround_setup    abc;
   dr     <- llvm_verify m "s20_doubleround"  [cr,rr] false␣
↪doubleround_setup    abc;
   s20    <- llvm_verify m "s20_hash"         [dr]    false salsa20_
↪setup        abc;
   s20e32 <- llvm_verify m "s20_expand32"     [s20]   true  salsa20_
↪expansion_32 abc;
   s20encrypt_63 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 63) abc;
   s20encrypt_64 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 64) abc;
   s20encrypt_65 <- llvm_verify m "s20_crypt32" [s20e32] true (s20_
↪encrypt32 65) abc;

   print "Done!";
};
```

11.18 Verifying Cryptol FFI functions

SAW has special support for verifying the correctness of Cryptol's `foreign` functions (https://galoisinc.github.io/cryptol/master/FFI.html), implemented in a language such as C which compiles to LLVM, provided that there exists a reference Cryptol implementation (https://galoisinc.github.io/cryptol/master/FFI.html#cryptol-implementation-of-foreign-functions) of the function as well. Since the way in which `foreign` functions are called is precisely specified by the Cryptol FFI, SAW is able to generate a `LLVMSetup ()` spec directly from the type of a Cryptol `foreign` function. This is done with the `llvm_ffi_setup` command, which is experimental and requires `enable_experimental;` to be run beforehand.

- `llvm_ffi_setup : Term -> LLVMSetup ()`

For instance, for the simple imported Cryptol foreign function `foreign add : [32] -> [32] -> [32]` we can obtain a `LLVMSetup` spec simply by writing

```
let add_setup = llvm_ffi_setup {{ add }};
```

which behind the scenes expands to something like

```
let add_setup = do {
  in0 <- llvm_fresh_var "in0" (llvm_int 32);
  in1 <- llvm_fresh_var "in1" (llvm_int 32);
  llvm_execute_func [llvm_term in0, llvm_term in1];
  llvm_return (llvm_term {{ add in0 in1 }});
};
```

11.18.1 Polymorphism

In general, Cryptol `foreign` functions can be polymorphic, with type parameters of kind `#`, representing e.g. the sizes of input sequences. However, any individual `LLVMSetup ()` spec only specifies the behavior of the LLVM function on inputs of concrete sizes. We handle this by allowing the argument term of `llvm_ffi_setup` to contain any necessary type arguments in addition to the Cryptol function name, so that the resulting term is monomorphic. The user can then define a parameterized specification simply as a SAWScript function in the usual way. For example, for a function `foreign f : {n, m} (fin n, fin m) => [n][32] -> [m][32]`, we can obtain a parameterized `LLVMSetup` spec by

```
let f_setup (n : Int) (m : Int) = llvm_ffi_setup {{ f`{n, m} }};
```

Note that the `Term` parameter that `llvm_ffi_setup` takes is restricted syntactically to the format described above (`{{ fun`{tyArg0, tyArg1, ..., tyArgN} }}`), and cannot be any arbitrary term.

11.18.2 Supported types

`llvm_ffi_setup` supports all Cryptol types that are supported by the Cryptol FFI, with the exception of `Integer`, `Rational`, `Z`, and `Float`. `Integer`, `Rational`, and `Z` are not supported since they are translated to `gmp` arbitrary-precision types which are hard for SAW to handle without additional overrides. There is no fundamental obstacle to supporting `Float`, and in fact `llvm_ffi_setup` itself does work with Cryptol floating point types, but the underlying functions such as `llvm_fresh_var` do not, so until that is implemented `llvm_ffi_setup` can generate a spec involving floating point types but it cannot actually be run.

Note also that for the time being only the `c` calling convention is supported. Support for the recently-added `abstract` calling convention has not been written yet. See issue #2546 (https://github.com/GaloisInc/saw-script/issues/2546).

11.18.3 Performing the verification

The resulting `LLVMSetup ()` spec can be used with the existing `llvm_verify` function to perform the actual verification. And the `LLVMSpec` output from that can be used as an override as usual for further compositional verification.

```
f_ov <- llvm_verify mod "f" [] true (f_setup 3 5) z3;
```

As with the Cryptol FFI itself, SAW does not manage the compilation of the C source implementations of `foreign` functions to LLVM bitcode. For the verification to be meaningful, is expected that the LLVM module passed to `llvm_verify` matches the compiled dynamic library actually used with the Cryptol interpreter. Alternatively, on x86_64 Linux, SAW can perform verification directly on the `.so` ELF file with the experimental `llvm_verify_x86` command.

11.19 Multi-Step Proofs and Loop Invariants with Cuts

The LLVM backend (and so far, only the LLVM backend) has the ability to divide a function under verification into two (or more) parts and apply assertions at the middle point as well as the beginning and end. This is called a "cut", after the similar operation in proof derivations, and we call the point of division a "cutpoint".

This feature is highly experimental and should be approached with caution.

At the code level, the cutpoint is a call to an empty placeholder function with a magic name. This does require modifying the code, although not in a way that has any material effect.

When verifying, SAW recognizes the magic name when loading the LLVM module and transforms the code by moving the rest of the code in the real function to the placeholder function and replacing it with a call to the placeholder.

Then one may write SAW specifications for the original function name (representing the whole original function) and for the placeholder function (representing only the second part of it).

The magic names SAW recognizes and applies this feature to are any function names beginning with `__cutpoint__`.

A simple example of a two-stage function, adapted from SAW's test suite (see `intTests/test_cutpoint`):

```
#include <stdlib.h>

extern size_t __cutpoint__add2(size_t *);

size_t add2(size_t x) {
  ++x;
  __cutpoint__add2(&x);
  ++x;
  return x;
}
```

This function adds two to its argument in separate steps, and we'll verify each half separately.

Note that the function `__cutpoint_add2` does not actually exist in this example, and does not need to for verification. It only exists to mark the cut. (If you wanted to run this code, you'd provide an empty implementation that does nothing.)

The corresponding SAW specifications are, first for `add2`:

```
let add2_spec = do {
  x <- llvm_fresh_var "x" (llvm_int 64);
  llvm_execute_func [llvm_term x];
```

(continues on next page)

(continued from previous page)

```
  llvm_return (llvm_term {{ x + 2 }});
};
```

And for `__cutpoint__add2`:

```
let cutpoint_add2_spec = do {
  p <- llvm_alloc (llvm_int 64);
  x <- llvm_fresh_var "x" (llvm_int 64);
  llvm_points_to p (llvm_term x);
  llvm_execute_func [p];
  llvm_return (llvm_term {{ x + 1 }});
};
```

There are several things to note here. First, the spec for `add2` spans the whole function: the return value it specifies is the overall return value, not the value of `x` at the cutpoint. There is no direct way to address the value of `x` at the cutpoint in the spec for the original function. The spec for the cutpoint function receives that value of `x`, and can write assertions about it, but there is in turn no easy way to get at the original value of `x` as of entry to `add2`. (The only way to do so is to change the code around to preserve it in another variable, which is almost always undesirable.) The verification will check that that the value of `x` as of the cutpoint satisfies the precondition given in the cutpoint function spec. However, for complex code these limitations may make writing that precondition difficult or impossible. (We might extend or alter the behavior in the future to make it possible. As noted above the feature is experimental; it has some sharp edges, of which this is one.)

Second, the spec for `cutpoint_add2` always covers the entire remainder of the function. If you use two cutpoints to divide a function into three parts, the first spec spans the whole function, the second spans the second and third part, and the third spans only the third part. There is (currently) no way to flip this around so the first spec spans just the first part of the function and the last spans the the whole function.

Third, the value of x is passed to the cutpoint through a pointer. This is necessary to make the propagation of the values through the transformed code work properly. It is also necessary to pass the values of all live variables, via pointers, to the cutpoint function, in the order they appear in the LLVM stack frame. Determining which variables these are and what order they appear in may require disassembling the LLVM bitcode (with `llvm-dis` or SAW's `:llvmdis` REPL command) and inspecting it. As of this writing, if any of this is not correct, SAW crashes deep inside the logic that performs the code transformation, and, alas, with a fairly mysterious message about being unable to find register values.

Now, to verify this function you do the following, assuming that `m` is the LLVM module:

```
cp <- llvm_verify m "__cutpoint__add2#add2" [] true cutpoint_add2_spec␣
↪z3;
llvm_verify m "add2" [cp] true add2_spec z3;
```

That is: first you verify the cutpoint function against its spec, then you use it as an override when verifying the original function.

Note the extra bit in the name in the first `llvm_verify` call: you must provide the name of the parent function after a `#`. SAW needs this to find the transformed code.

11.19.1 Applying Cuts Under Conditionals

If you place a cutpoint in a block that is conditionally executed (e.g. under an if whose control is not constant) the cutpoint function still continues the entire remainder of the function. Thus, the spec for the cutpoint function must include the behavior of all the code after the conditional it's in closes, but need not reason about the other branch of the conditional at all.

If you place cutpoints in both sides of a conditional, both cutpoint specs must cover the entire remainder of the function. However, in some cases that can be more specific about its behavior than the general case would be. (The spec for the original function must still give some return value, but it need not necessarily be fully specific.)

This can sometimes be helpful for handling functions with diamond-shaped control flow.

You cannot, unfortunately, splice the execution back together after the conditionals have ended.

11.19.2 Using Cut with Loops

While dividing a large or complicated sequential function into multiple parts can be useful in its own right, the real purpose of the cut feature is to allow reasoning about loops by providing loop invariants.

When you place a cutpoint in the condition of a loop, the cutpoint function becomes the rest of the original function… which is to say, it executes the loop body and then comes back to the loop condition. In particular, the loop is converted into recursion and the recursive call becomes another call to the cutpoint function. This makes the precondition of the cutpoint function a loop invariant.

It also means you can escape SAW's restriction about loops being concretely bounded. The trick is to first assume the spec for the cutpoint function, then use that as an override when proving it. The proof will check that the loop body correctly assumes and then restores the loop invariant, and the override using the assumed version will let SAW conclude that the loop invariant remains true after the recursive call without having to execute through it again.

However, beware: this technique only provides partial correctness. It does not allow reasoning about the termination of the loop. It allows you to prove that *if* the loop terminates, the the original function returns according to the spec. If the loop doesn't terminate, it doesn't terminate.

Here is an example, also adapted from the test suite:

```
#include <stdlib.h>

extern size_t __cutpoint__inv(size_t*, size_t*, size_t*) __attribute__
↪((noduplicate));
```

(continues on next page)

(continued from previous page)

```
size_t count_n(size_t n) {
  size_t c = 0;
  for (size_t i = 0; __cutpoint__inv(&n, &c, &i), i < n; ++i) {
    ++c;
  }
  return c;
}
```

This rather naively counts up to `n`. SAW can't handle this function by default, or at least not in general: you can prove it works for any given value of `n`, but proving a general theorem about it for all `n` requires it to try the loop all `n` times. (Because `n` is a bitvector, that's at least not infinitely many times; however, because it's a 64-bit bitvector, it is too many to check in any effective amount of time.)

Instead we can write this spec for the cutpoint:

```
let ptr_to_fresh n ty = do {
  p <- llvm_alloc ty;
  x <- llvm_fresh_var n ty;
  llvm_points_to p (llvm_term x);
  return (p, x);
};

let inv_spec = do {
  (pn, n) <- ptr_to_fresh "n" (llvm_int 64);
  (pc, c) <- ptr_to_fresh "c" (llvm_int 64);
  (pi, i) <- ptr_to_fresh "i" (llvm_int 64);
  llvm_precond {{ 0 <= i /\ i <= n }};
  llvm_execute_func [pn, pc, pi];
  llvm_return (llvm_term {{ c + (n - i) }});
};
```

This is about what one would expect: it takes all three of the live variables as pointer arguments, and the loop invariant is the loop condition extended to the case where the loop terminates.

The return value seems slightly unexpected; surely the return value should be just `n`? If you try that, it doesn't work. But that's because the loop invariant isn't strong enough; it doesn't say anything about the relationship of `c` and `i`.

What's here is true even if they are out of sync: when the loop terminates (if it terminates), `i` is equal to `n` and we return `c`, whatever it is, which is what the C code does.

One can also write this spec:

```
let inv_spec = do {
  (pn, n) <- ptr_to_fresh "n" (llvm_int 64);
  (pc, c) <- ptr_to_fresh "c" (llvm_int 64);
  (pi, i) <- ptr_to_fresh "i" (llvm_int 64);
  llvm_precond {{ 0 <= i /\ i <= n /\ c == i }};
  llvm_execute_func [pn, pc, pi];
  llvm_return (llvm_term {{ n }});
};
```

and this will also verify.

However, in either case this spec for the original function will still verify successfully:

```
let count_n_spec = do {
  n <- llvm_fresh_var "n" (llvm_int 64);
  llvm_execute_func [llvm_term n];
  llvm_return (llvm_term n);
};
```

Verifying this function only reaches the loop when `c` and `n` are equal, and that's sufficient for SAW to show that, because `i` is equal to `n` when the loop terminates (if it terminates), `c` is also equal to `n`.

Choosing the correct loop invariant is a sometimes subtle art. (It is not in any way SAW-specific and much has been written about it elsewhere.)

Note however that as with the sequential usage, the whole rest of the function after the loop belongs to the cutpoint function; if there is code after the loop, the cutpoint spec needs to account for it.

This is how you actually run the verification:

```
inv <- llvm_unsafe_assume_spec m "__cutpoint__inv#count_n" inv_spec;
llvm_verify m "__cutpoint__inv#count_n" [inv] false inv_spec abc;
llvm_verify m "count_n" [inv] false count_n_spec abc;
```

As described above, first you assume the spec for the cutpoint function, then you verify it using the assumed version as an override for the recursive call. (This is a proof by induction: the code leading up to the loop is the base case that establishes the inductive hypothesis, which is the loop invariant; then the loop body is the inductive case. Then for any finite number of iterations the loop invariant holds, and so it holds when the loop terminates… if it terminates.)

Notice that the cutpoint function is labeled `noduplicate`; this prevents LLVM from doing optimizations that might break SAW's ability to do the cut transform.

CHAPTER 12

Bisimulation Prover

SAW contains a bisimulation prover to prove that two terms simulate each other. This prover allows users to prove that two terms executing in lockstep satisfy some relations over the state of each circuit and their outputs. This type of proof is useful in demonstrating the eventual equivalence of two circuits, or of a circuit and a functional specification. SAW enables these proofs with the experimental `prove_bisim` command:

```
prove_bisim : ProofScript () -> [BisimTheorem] -> Term -> Term -> Term -
↪> Term -> TopLevel BisimTheorem
```

When invoking `prove_bisim strat theorems srel orel lhs rhs`, the arguments represent the following:

1. `strat`: A proof strategy to use during verification.
2. `theorems`: A list of already proven bisimulation theorems.
3. `srel`: A state relation between `lhs` and `rhs`. This relation must have the type `lhsState -> rhsState -> Bit`. The relation's first argument is `lhs`'s state prior to execution. The relation's second argument is `rhs`'s state prior to execution. `srel` then returns a `Bit` indicating whether the two arguments satisfy the bisimulation's state relation.
4. `orel`: An output relation between `lhs` and `rhs`. This relation must have the type `(lhsState, output) -> (rhsState, output) -> Bit`. The relation's first argument is a pair consisting of `lhs`'s state and output following execution. The relation's second argument is a pair consisting of `rhs`'s state and output following execution. `orel` then returns a `Bit` indicating whether the two arguments satisfy the bisimulation's output relation.
5. `lhs`: A term that simulates `rhs`. `lhs` must have the type `(lhsState, input) ->`

(lhsState, output). The first argument to lhs is a tuple containing the internal state of lhs, as well as the input to lhs. lhs returns a tuple containing its internal state after execution, as well as its output.

6. rhs: A term that simulates lhs. rhs must have the type (rhsState, input) -> (rhsState, output). The first argument to rhs is a tuple containing the internal state of rhs, as well as the input to rhs. rhs returns a tuple containing its internal state after execution, as well as its output.

On success, prove_bisim returns a BisimTheorem that can be used in future bisimulation proofs to enable compositional bisimulation proofs. On failure, prove_bisim will abort.

12.1 Bisimulation Example

This section walks through an example proving that the Cryptol implementation of an AND gate that makes use of internal state and takes two cycles to complete is equivalent to a pure function that computes the logical AND of its inputs in one cycle. First, we define the implementation's state type:

```
type andState = { loaded : Bit, origX : Bit, origY : Bit }
```

andState is a record type with three fields:

1. loaded: A Bit indicating whether the input to the AND gate has been loaded into the state record.
2. origX: A Bit storing the first input to the AND gate.
3. origY: A Bit storing the second input to the AND gate.

Now, we define the AND gate's implementation:

```
andImp : (andState, (Bit, Bit)) -> (andState, (Bit, Bit))
andImp (s, (x, y)) =
  if s.loaded /\ x == s.origX /\ y == s.origY
  then (s, (True, s.origX && s.origY))
  else ({ loaded = True, origX = x, origY = y }, (False, 0))
```

andImp takes a tuple as input where the first field is an andState holding the gate's internal state, and second field is a tuple containing the inputs to the AND gate. andImp returns a tuple consisting of the updated andState and the gate's output. The output is a tuple where the first field is a ready bit that is 1 when the second field is ready to be read, and the second field is the result of gate's computation.

andImp takes two cycles to complete:

1. The first cycle loads the inputs into its state's origX and origY fields and sets loaded to True. It sets both of its output bits to 0.

2. The second cycle uses the stored input values to compute the logical AND. It sets its ready bit to `1` and its second output to the logical AND result.

So long as the inputs remain fixed after the second cycle, `andImp`'s output remains unchanged. If the inputs change, then `andImp` restarts the computation (even if the inputs change between the first and second cycles).

Next, we define the pure function we'd like to prove `andImp` bisimilar to:

```
andSpec : ((), (Bit, Bit)) -> ((), (Bit, Bit))
andSpec (_, (x, y)) = ((), (True, x && y))
```

`andSpec` takes a tuple as input where the first field is `()`, indicating that `andSpec` is a pure function without internal state, and the second field is a tuple containing the inputs to the AND function. `andSpec` returns a tuple consisting of `()` (again, because `andSpec` is stateless) and the function's output. Like `andImp`, the output is a tuple where the first field is a ready bit that is `1` when the second field is ready to be read, and the second field is the result of the function's computation.

`andSpec` completes in a single cycle, and as such its ready bit is always `1`. It computes the logical AND directly on the function's inputs using Cryptol's `(&&)` operator.

Next, we define a state relation over `andImp` and `andSpec`:

```
andStateRel : andState -> () -> Bit
andStateRel _ () = True
```

`andStateRel` takes two arguments:

1. An `andState` for `andImp`.
2. An empty state (`()`) for `andSpec`.

`andStateRel` returns a `Bit` indicating whether the relation is satisified. In this case, `andStateRel` always returns `True` because `andSpec` is stateless and therefore the state relation permits `andImp` to accept any state.

Lastly, we define a relation over `andImp` and `andSpec`:

```
andOutputRel : (andState, (Bit, Bit)) -> ((), (Bit, Bit)) -> Bit
andOutputRel (s, (impReady, impO)) ((), (_, specO)) =
  if impReady then impO == specO else True
```

`andOutputRel` takes two arguments:

1. A return value from `andImp`. Specifically, a pair consisting of an `andState` and a pair containing a ready bit and result of the logical AND.
2. A return value from `andSpec`. Specifically, a pair consisting of an empty state `()` and a pair containing a ready bit and result of the logical AND.

`andOutputRel` returns a `Bit` indicating whether the relation is satisfied. It considers the relation satisfied in two ways:

1. If `andImp`'s ready bit is set, the relation is satisfied if the output values `impO` and `specO` from `andImp` and `andSpec` respectively are equivalent.
2. If `andImp`'s ready bit is not set, the relation is satisfied.

Put another way, the relation is satisfied if the end result of `andImp` and `andSpec` are equivalent. The relation permits intermediate outputs to differ.

We can verify that this relation is always satisfied–and therefore the two terms are bisimilar–by using `prove_bisim`:

```
import "And.cry";
enable_experimental;

and_bisim <- prove_bisim z3 [] {{ andStateRel }} {{ andOutputRel }} {{␣
↪andImp }} {{ andSpec }};
```

Upon running this script, SAW prints:

```
Successfully proved bisimulation between andImp and andSpec
```

12.1.1 Building a NAND gate

We can make the example more interesting by reusing components to build a NAND gate. We first define a state type for the NAND gate implementation that contains `andImp`'s state. This NAND gate will not need any additional state, so we will define a type `nandState` that is equal to `andState`:

```
type nandState = andState
```

Now, we define an implementation `nandImp` that calls `andImp` and negates the result:

```
nandImp : (nandState, (Bit, Bit)) -> (nandState, (Bit, Bit))
nandImp x = (s, (andReady, ~andRes))
  where
    (s, (andReady, andRes)) = andImp x
```

Note that `nandImp` is careful to preserve the ready status of `andImp`. Because `nandImp` relies on `andImp`, it also takes two cycles to compute the logical NAND of its inputs.

Next, we define a specification `nandSpec` in terms of `andSpec`:

```
nandSpec : ((), (Bit, Bit)) -> ((), (Bit, Bit))
nandSpec (_, (x, y)) = ((), (True, ~ (andSpec ((), (x, y))).1.1))
```

As with `andSpec`, `nandSpec` is pure and computes its result in a single cycle.

Next, we define a state relation over `nandImp` and `nandSpec`:

```
nandStateRel : andState -> () -> Bit
nandStateRel _ () = True
```

As with `andStateRel`, this state relation is always `True` because `nandSpec` is stateless.

Lastly, we define an output relation indicating that `nandImp` and `nandSpec` produce equivalent results once `nandImp`'s ready bit is `1`:

```
nandOutputRel : (nandState, (Bit, Bit)) -> ((), (Bit, Bit)) -> Bit
nandOutputRel (s, (impReady, impO)) ((), (_, specO)) =
  if impReady then impO == specO else True
```

To prove that `nandImp` and `nandSpec` are bisimilar, we again use `prove_bisim`. This time however, we can reuse the bisimulation proof for the AND gate by including it in the `theorems` paramter for `prove_bisim`:

```
prove_bisim z3 [and_bisim] {{ nandStateRel }} {{ nandOutputRel }} {{␣
↪nandImp }} {{ nandSpec }};
```

Upon running this script, SAW prints:

```
Successfully proved bisimulation between nandImp and nandSpec
```

12.2 Understanding the proof goals

While not necessary for simple proofs, more advanced proofs may require inspecting proof goals. `prove_bisim` generates and attempts to solve the following proof goals:

```
OUTPUT RELATION THEOREM:
  forall s1 s2 in.
    srel s1 s2 -> orel (lhs (s1, in)) (rhs (s2, in))

STATE RELATION THEOREM:
  forall s1 s2 out1 out2.
    orel (s1, out1) (s2, out2) -> srel s1 s2
```

where the variables in the `forall`s are:

- `s1`: Initial state for `lhs`
- `s2`: Initial state for `rhs`
- `in`: Input value to `lhs` and `rhs`
- `out1`: Initial output value for `lhs`

- `out2`: Initial output value for `rhs`

The `STATE RELATION THEOREM` verifies that the output relation properly captures the guarantees of the state relation. The `OUTPUT RELATION THEOREM` verifies that if `lhs` and `rhs` are executed with related states, then the result of that execution is also related. These two theorems together guarantee that the terms simulate each other.

When using composition, `prove_bisim` also generates and attempts to solve the proof goal below for any successfully applied `BisimTheorem` in the `theorems` list:

```
COMPOSITION SIDE CONDITION:
  forall g_lhs_s g_rhs_s.
    g_srel g_lhs_s g_rhs_s -> f_srel f_lhs_s f_rhs_s
    where
      f_lhs_s = extract_inner_state g_lhs g_lhs_s f_lhs
      f_rhs_s = extract_inner_state g_rhs g_rhs_s f_rhs
```

where `g_lhs` is an outer term containing a call to an inner term `f_lhs` represented by a `BisimTheorem` and `g_rhs` is an outer term containing a call to an inner term `f_rhs` represented by the same `BisimTheorem`. The variables in `COMPOSITION SIDE CONDITION` are:

- `extract_inner_state x x_s y`: A helper function that takes an outer term `x`, an outer state `x_s`, and an inner term `y`, and returns the inner state of `x_s` that `x` passes to `y`.
- `g_lhs_s`: The state for `g_lhs`
- `g_rhs_s`: The state for `g_rhs`
- `g_srel`: The state relation for `g_lhs` and `g_rhs`
- `f_srel`: The state relation for `f_lhs` and `f_rhs`
- `f_lhs_s`: The state for `f_lhs`, as represented in `g_lhs_s` (extracted using `extract_inner_state`).
- `f_rhs_s`: The state for `f_rhs`, as represented in `g_rhs_s` (extracted using `extract_inner_state`).

The `COMPOSITION SIDE CONDITION` exists to verify that the terms in the bisimulation relation properly set up valid states for subterms they contain.

12.3 Limitations

For now, the `prove_bisim` command has a couple limitations:

- `lhs` and `rhs` must be named functions. This is because `prove_bisim` uses these names to perform substitution when making use of compositionality.
- Each subterm present in the list of bisimulation theorems already proven may be invoked at most once in `lhs` or `rhs`. That is, if some function `g_lhs` calls `f_lhs`, and `prove_bisim`

is invoked with a `BisimTheorem` proving that `f_lhs` is bisimilar to `f_rhs`, then `g_lhs` may call `f_lhs` at most once.

CHAPTER 13

Transforming Term Values

The three primary functions of SAW are *extracting* models (`Term` values) from programs, *transforming* those models, and *proving* properties about models using external provers. We've seen how to construct `Term` values from a range of sources. Now we show how to use the various term transformation features available in SAW.

13.1 Rewriting

Rewriting a `Term` consists of applying one or more *rewrite rules* to it, resulting in a new `Term`. A rewrite rule in SAW can be specified in multiple ways:

- as the definition of a function that can be unfolded,
- as a term of Boolean type (or a function returning a Boolean) that is an equality statement, and
- as a term of *equality type* with a body that encodes a proof that the equality in the type is valid.

In each case the term logically consists of two sides and describes a way to transform the left side into the right side. Each side may contain variables (bound by enclosing lambda expressions) and is therefore a *pattern* which can match any term in which each variable represents an arbitrary sub-term. The left-hand pattern describes a term to match (which may be a sub-term of the full term being rewritten), and the right-hand pattern describes a term to replace it with. Any variable in the right-hand pattern must also appear in the left-hand pattern and will be instantiated with whatever sub-term matched that variable in the original term.

For example, say we have the following Cryptol function:

```
\(x:[8]) -> (x * 2) + 1
```

We might for some reason want to replace multiplication by a power of two with a shift. We can describe this replacement using an equality statement in Cryptol (a rule of form 2 above):

```
\(y:[8]) -> (y * 2) == (y << 1)
```

Interpreting this as a rewrite rule, it says that for any 8-bit vector (call it `y` for now), we can replace `y * 2` with `y << 1`. Using this rule to rewrite the earlier expression would then yield:

```
\(x:[8]) -> (x << 1) + 1
```

The general philosophy of rewriting is that the left and right patterns, while syntactically different, should be semantically equivalent. Therefore, applying a set of rewrite rules should not change the fundamental meaning of the term being rewritten. SAW is particularly focused on the task of proving that some logical statement expressed as a `Term` is always true. If that is in fact the case, then the entire term can be replaced by the term `True` without changing its meaning. The rewriting process can in some cases, by repeatedly applying rules that themselves are known to be valid, reduce a complex term entirely to `True`, which constitutes a proof of the original statement. In other cases, rewriting can simplify terms before sending them to external automated provers that can then finish the job. Sometimes this simplification can help the automated provers run more quickly, and sometimes it can help them prove things they would otherwise be unable to prove by applying reasoning steps (rewrite rules) that are not available to the automated provers.

In practical use, rewrite rules can be aggregated into `Simpset` values in SAWScript. A few predefined `Simpset` values exist:

- `empty_ss : Simpset` is the empty set of rules. Rewriting with it should have no effect, but it is useful as an argument to some of the functions that construct larger `Simpset` values.
- `basic_ss : Simpset` is a collection of rules that are useful in most proof scripts.
- `cryptol_ss : () -> Simpset` includes a collection of Cryptol-specific rules. Some of these simplify away the abstractions introduced in the translation from Cryptol to SAWCore, which can be useful when proving equivalence between Cryptol and non-Cryptol code. Leaving these abstractions in place is appropriate when comparing only Cryptol code, however, so `cryptol_ss` is not included in `basic_ss`.

The following function can apply a `Simpset`:

- `rewrite : Simpset -> Term -> Term` applies a `Simpset` to an existing `Term` to produce a new `Term`.

To make this more concrete, we examine how the rewriting example sketched above, to convert multiplication into shift, can work in practice. We simplify everything with `cryptol_ss` as we go along so that the `Term`s don't get too cluttered. First, we declare the term to be transformed:

```
sawscript> let term = rewrite (cryptol_ss ()) {{ \(x:[8]) -> (x * 2) +␣
↪1 }}
sawscript> print_term term
\(x : Prelude.Vec 8 Prelude.Bool) ->
  Prelude.bvAdd 8 (Prelude.bvMul 8 x (Prelude.bvNat 8 2))
    (Prelude.bvNat 8 1)
```

Next, we declare the rewrite rule:

```
sawscript> let rule = rewrite (cryptol_ss ()) {{ \(y:[8]) -> (y * 2) ==␣
↪(y << 1) }}
sawscript> print_term rule
let { x@1 = Prelude.Vec 8 Prelude.Bool
    }
 in \(y : x@1) ->
      Cryptol.ecEq x@1 (Cryptol.PCmpWord 8)
        (Prelude.bvMul 8 y (Prelude.bvNat 8 2))
        (Prelude.bvShiftL 8 Prelude.Bool 1 Prelude.False y
           (Prelude.bvNat 1 1))
```

The primary interface to rewriting uses the `Theorem` type instead of the `Term` type, as shown in the signatures for `addsimp` and `addsimps`.

- `addsimp : Theorem -> Simpset -> Simpset` adds a single `Theorem` to a `Simpset`.
- `addsimps : [Theorem] -> Simpset -> Simpset` adds several `Theorem` values to a `Simpset`.

A `Theorem` is essentially a `Term` that is proven correct in some way. In general, a `Theorem` can be any statement, and may not be useful as a rewrite rule. However, if it has an appropriate shape it can be used for rewriting. In the *"Proofs about Terms"* section, we'll describe how to construct `Theorem` values from `Term` values.

For the time being, we'll assume we've proved our `rule` term correct in some way, and have a `Theorem` named `rule_thm`.

Finally, we apply the rule to the target term:

```
sawscript> let result = rewrite (addsimp rule_thm empty_ss) term
sawscript> print_term result
\(x : Prelude.Vec 8 Prelude.Bool) ->
  Prelude.bvAdd 8
    (Prelude.bvShiftL 8 Prelude.Bool 1 Prelude.False x
       (Prelude.bvNat 1 1))
    (Prelude.bvNat 8 1)
```

In the absence of user-constructed `Theorem` values, there are some additional built-in rules that are

not included in either `basic_ss` and `cryptol_ss` because they are not always beneficial, but that can sometimes be helpful or essential. The `cryptol_ss` simpset includes rewrite rules to unfold all definitions in the `Cryptol` SAWCore module, but does not include any of the terms of equality type.

- `add_cryptol_defs : [String] -> Simpset -> Simpset` adds unfolding rules for functions with the given names from the SAWCore `Cryptol` module to the given `Simpset`.
- `add_cryptol_eqs : [String] -> Simpset -> Simpset` adds the terms of equality type with the given names from the SAWCore `Cryptol` module to the given `Simpset`.
- `add_prelude_defs : [String] -> Simpset -> Simpset` adds unfolding rules from the SAWCore `Prelude` module to a `Simpset`.
- `add_prelude_eqs : [String] -> Simpset -> Simpset` adds equality-typed terms from the SAWCore `Prelude` module to a `Simpset`.

Finally, it's possible to construct a theorem from an arbitrary SAWCore expression (rather than a Cryptol expression), using the `core_axiom` function.

- `core_axiom : String -> Theorem` creates a `Theorem` from a `String` in SAWCore syntax. Any `Theorem` introduced by this function is assumed to be correct, so use it with caution.

13.2 Folding and Unfolding

A SAWCore term can be given a name using the `define` function, and is then by default printed as that name alone. A named subterm can be "unfolded" so that the original definition appears again.

- `define : String -> Term -> TopLevel Term`
- `unfold_term : [String] -> Term -> Term`

For example:

```
sawscript> let t = {{ 0x22 }}
sawscript> print_term t
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
sawscript> t' <- define "t" t
sawscript> print_term t'
t
sawscript> let t'' = unfold_term ["t"] t'
sawscript> print_term t''
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
```

This process of folding and unfolding is useful both to make large terms easier for humans to work with and to make automated proofs more tractable. We'll describe the latter in more detail when we discuss interacting with external provers.

In some cases, folding happens automatically when constructing Cryptol expressions. Consider the following example:

```
sawscript> let t = {{ 0x22 }}
sawscript> print_term t
Cryptol.ecNumber (Cryptol.TCNum 34) (Prelude.Vec 8 Prelude.Bool)
  (Cryptol.PLiteralSeqBool (Cryptol.TCNum 8))
sawscript> let {{ t' = 0x22 }}
sawscript> print_term {{ t' }}
t'
```

This illustrates that a bare expression in Cryptol braces gets translated directly to a SAWCore term. However, a Cryptol *definition* gets translated into a *folded* SAWCore term. In addition, because the second definition of `t` occurs at the Cryptol level, rather than the SAWScript level, it is visible only inside Cryptol braces. Definitions imported from Cryptol source files are also initially folded and can be unfolded as needed.

13.3 Other Built-in Transformation and Inspection Functions

In addition to the `Term` transformation functions described so far, a variety of others also exist.

- `beta_reduce_term : Term -> Term` takes any sub-expression of the form `(\x -> t) v` in the given `Term` and replaces it with a transformed version of `t` in which all instances of `x` are replaced by `v`.
- `replace : Term -> Term -> Term -> TopLevel Term` replaces arbitrary subterms. A call to `replace x y t` replaces any instance of `x` inside `t` with `y`.

Assessing the size of a term can be particularly useful during benchmarking. SAWScript provides two mechanisms for this.

- `term_size : Term -> Int` calculates the number of nodes in the Directed Acyclic Graph (DAG) representation of a `Term` used internally by SAW. This is the most appropriate way of determining the resource use of a particular term.
- `term_tree_size : Term -> Int` calculates how large a `Term` would be if it were represented by a tree instead of a DAG. This can, in general, be much, much larger than the number returned by `term_size`, and serves primarily as a way of assessing, for a specific term, how much benefit there is to the term sharing used by the DAG representation.

Finally, there are a few commands related to the internal SAWCore type of a `Term`.

- `check_term : Term -> TopLevel ()` checks that the internal structure of a `Term` is well-formed and that it passes all of the rules of the SAWCore type checker.
- `type : Term -> Type` returns the type of a particular `Term`, which can then be used to, for example, construct a new fresh variable with `fresh_symbolic`.

13.4 Loading and Storing Terms

Most frequently, `Term` values in SAWScript come from Cryptol, JVM, or LLVM programs, or some transformation thereof. However, it is also possible to obtain them from various other sources.

- `parse_core : String -> Term` parses a `String` containing a term in SAWCore syntax, returning a `Term`.
- `read_core : String -> TopLevel Term` is like `parse_core`, but obtains the text from the given file and expects it to be in the simpler SAWCore external representation format, rather than the human-readable syntax shown so far.
- `read_aig : String -> TopLevel Term` returns a `Term` representation of an And-Inverter-Graph (AIG) file in AIGER format.
- `read_bytes : String -> TopLevel Term` reads a constant sequence of bytes from a file and represents it as a `Term`. Its result will always have Cryptol type `[n][8]` for some `n`.

It is also possible to write `Term` values into files in various formats, including: AIGER (`write_aig`), CNF (`write_cnf`), SAWCore external representation (`write_core`), and SMT-Lib version 2 (`write_smtlib2`).

- `write_aig : String -> Term -> TopLevel ()`
- `write_cnf : String -> Term -> TopLevel ()`
- `write_core : String -> Term -> TopLevel ()`
- `write_smtlib2 : String -> Term -> TopLevel ()`

CHAPTER 14

Proofs about Terms

The goal of SAW is to facilitate proofs about the behavior of programs. It may be useful to prove some small fact to use as a rewrite rule in later proofs, but ultimately these rewrite rules come together into a proof of some higher-level property about a software system.

Whether proving small lemmas (in the form of rewrite rules) or a top-level theorem, the process builds on the idea of a *proof script* that is run by one of the top level proof commands.

- `prove_print : ProofScript () -> Term -> TopLevel Theorem` takes a proof script (which we'll describe next) and a `Term`. The `Term` should be of function type with a return value of `Bool` (`Bit` at the Cryptol level). It will then use the proof script to attempt to show that the `Term` returns `True` for all possible inputs. If it is successful, it will print `Valid` and return a `Theorem`. If not, it will abort.
- `sat_print : ProofScript () -> Term -> TopLevel ()` is similar except that it looks for a *single* value for which the `Term` evaluates to `True` and prints out that value, returning nothing.
- `prove_core : ProofScript () -> String -> TopLevel Theorem` proves and returns a `Theorem` from a string in SAWCore syntax.

14.1 Automated Tactics

The simplest proof scripts just specify the automated prover to use. The `ProofScript` values `abc` and `z3` select the ABC and Z3 theorem provers, respectively, and are typically good choices.

For example, combining `prove_print` with `abc`:

```
sawscript> t <- prove_print abc {{ \(x:[8]) -> x+x == x*2 }}
Valid
sawscript> t
Theorem (let { x@1 = Prelude.Vec 8 Prelude.Bool
      x@2 = Cryptol.TCNum 8
      x@3 = Cryptol.PArithSeqBool x@2
    }
 in (x : x@1)
-> Prelude.EqTrue
     (Cryptol.ecEq x@1 (Cryptol.PCmpSeqBool x@2)
        (Cryptol.ecPlus x@1 x@3 x x)
        (Cryptol.ecMul x@1 x@3 x
           (Cryptol.ecNumber (Cryptol.TCNum 2) x@1
              (Cryptol.PLiteralSeqBool x@2)))))
```

Similarly, `sat_print` will show that the function returns `True` for one specific input (which it should, since we already know it returns `True` for all inputs):

```
sawscript> sat_print abc {{ \(x:[8]) -> x+x == x*2 }}
Sat: [x = 0]
```

In addition to these, the `bitwuzla`, `boolector`, `cvc4`, `cvc5`, `mathsat`, and `yices` provers are available. The internal decision procedure `rme`, short for Reed-Muller Expansion, is an automated prover that works particularly well on the Galois field operations that show up, for example, in AES.

In more complex cases, some pre-processing can be helpful or necessary before handing the problem off to an automated prover. The pre-processing can involve rewriting, beta reduction, unfolding, the use of provers that require slightly more configuration, or the use of provers that do very little real work.

14.2 Proof Script Diagnostics

During development of a proof, it can be useful to print various information about the current goal. The following tactics are useful in that context.

- `print_goal : ProofScript ()` prints the entire goal in SAWCore syntax.
- `print_goal_consts : ProofScript ()` prints a list of unfoldable constants in the current goal.
- `print_goal_depth : Int -> ProofScript ()` takes an integer argument, `n`, and prints the goal up to depth `n`. Any elided subterms are printed with a `...` notation.
- `print_goal_size : ProofScript ()` prints the number of nodes in the DAG representation of the goal.

14.3 Rewriting in Proof Scripts

One of the key techniques available for completing proofs in SAWScript is the use of rewriting or transformation. The following commands support this approach.

- `simplify : Simpset -> ProofScript ()` works just like `rewrite`, except that it works in a `ProofScript` context and implicitly transforms the current (unnamed) goal rather than taking a `Term` as a parameter.
- `goal_eval : ProofScript ()` will evaluate the current proof goal to a first-order combination of primitives.
- `goal_eval_unint : [String] -> ProofScript ()` works like `goal_eval` but avoids expanding or simplifying the given names.

14.4 Other Transformations

Some useful transformations are not easily specified using equality statements, and instead have special tactics.

- `beta_reduce_goal : ProofScript ()` works like `beta_reduce_term` but on the current goal. It takes any sub-expression of the form `(\x -> t) v` and replaces it with a transformed version of `t` in which all instances of `x` are replaced by `v`.
- `unfolding : [String] -> ProofScript ()` works like `unfold_term` but on the current goal.

Using `unfolding` is mostly valuable for proofs based entirely on rewriting, since the default behavior for automated provers is to unfold everything before sending a goal to a prover. However, with some provers it is possible to indicate that specific named subterms should be represented as uninterpreted functions.

- `unint_bitwuzla : [String] -> ProofScript ()`
- `unint_cvc4 : [String] -> ProofScript ()`
- `unint_cvc5 : [String] -> ProofScript ()`
- `unint_yices : [String] -> ProofScript ()`
- `unint_z3 : [String] -> ProofScript ()`

The list of `String` arguments in these cases indicates the names of the subterms to leave folded, and therefore present as uninterpreted functions to the prover. To determine which folded constants appear in a goal, use the `print_goal_consts` function described above.

Ultimately, we plan to implement a more generic tactic that leaves certain constants uninterpreted in whatever prover is ultimately used (provided that uninterpreted functions are expressible in the prover).

Note that each of the `unint_*` tactics have variants that are prefixed with `sbv_` and `w4_`. The `sbv_`-prefixed tactics make use of the SBV library to represent and solve SMT queries:

- `sbv_unint_bitwuzla : [String] -> ProofScript ()`
- `sbv_unint_cvc4 : [String] -> ProofScript ()`
- `sbv_unint_cvc5 : [String] -> ProofScript ()`
- `sbv_unint_yices : [String] -> ProofScript ()`
- `sbv_unint_z3 : [String] -> ProofScript ()`

The `w4_`-prefixed tactics make use of the What4 library instead of SBV:

- `w4_unint_bitwuzla : [String] -> ProofScript ()`
- `w4_unint_cvc4 : [String] -> ProofScript ()`
- `w4_unint_cvc5 : [String] -> ProofScript ()`
- `w4_unint_yices : [String] -> ProofScript ()`
- `w4_unint_z3 : [String] -> ProofScript ()`

In most specifications, the choice of SBV versus What4 is not important, as both libraries are broadly compatible in terms of functionality. There are some situations where one library may outpeform the other, however, due to differences in how each library represents certain SMT queries. There are also some experimental features that are only supported with What4 at the moment, such as `enable_lax_loads_and_stores`.

14.5 Caching Solver Results

SAW has the capability to cache the results of tactics which call out to automated provers. This can save a considerable amount of time in cases such as proof development and CI, where the same proof scripts are often run repeatedly without changes.

This caching is available for all tactics which call out to automated provers at runtime: `abc`, `boolector`, `cvc4`, `cvc5`, `mathsat`, `yices`, `z3`, `rme`, and the family of `unint` tactics described in the previous section.

When solver caching is enabled and one of the tactics mentioned above is encountered, if there is already an entry in the cache corresponding to the call then the cached result is used, otherwise the appropriate solver is queried, and the result saved to the cache. Entries are indexed by a SHA256 hash of the exact query to the solver (ignoring variable names), any options passed to the solver, and the names and full version strings of all the solver backends involved (e.g. ABC and SBV for the `abc` tactic). This ensures cached results are only used when they would be identical to the result of actually running the tactic.

The simplest way to enable solver caching is to set the environment variable *`SAW_SOLVER_CACHE_PATH`*. With this environment variable set, `saw` and `saw-remote-api`

will automatically keep an LMDB (http://www.lmdb.tech/doc/) database at the given path containing the solver result cache. Setting this environment variable globally therefore creates a global, concurrency-safe solver result cache used by all newly created `saw` or `saw-remote-api` processes. Note that when this environment variable is set, SAW does not create a cache at the specified path until it is actually needed.

There are also a number of SAW commands related to solver caching.

- `set_solver_cache_path` is like setting `SAW_SOLVER_CACHE_PATH` for the remainder of the current session, but opens an LMDB database at the specified path immediately. If a cache is already in use in the current session (i.e. through a prior call to `set_solver_cache_path` or through `SAW_SOLVER_CACHE_PATH` being set and the cache being used at least once) then all entries in the cache already in use will be copied to the new cache being opened.
- `set_solver_cache_timeout` sets the cache's timeout (in microseconds) used for database lookups and inserts. The default timeout value is 2,000,000 microseconds (2 seconds). This is a reasonably large timeout for most cache operations, but it may be convenient to increase this timeout for especially large proof goals.
- `clean_mismatched_versions_solver_cache` will remove all entries in the solver result cache which were created using solver backend versions which do not match the versions in the current environment. This can be run after an update to clear out any old, unusable entries from the solver cache. This command can also be run directly from the command line through the `--clean-mismatched-versions-solver-cache` command-line option.
- `print_solver_cache` prints to the console all entries in the cache whose SHA256 hash keys start with the given hex string. Providing an empty string results in all entries in the cache being printed.
- `print_solver_cache_stats` prints to the console statistics including the size of the solver cache, where on disk it is stored, and some counts of how often it has been used during the current session.

For performing more complicated database operations on the set of cached results, the file `solver_cache.py` is provided with the Python bindings of the SAW Remote API. This file implements a general-purpose Python interface for interacting with the LMDB databases kept by SAW for solver caching.

Below is an example of using solver caching with `saw -v Debug`. Only the relevant output is shown, the rest abbreviated with "…".

```
sawscript> set_solver_cache_path "example.cache"
sawscript> prove_print z3 {{ \(x:[8]) -> x+x == x*2 }}
[22:13:00.832] Caching result: d1f5a76e7a0b7c01 (SBV 9.2, Z3 4.8.7 - 64␣
↪bit)
...
sawscript> prove_print z3 {{ \(new:[8]) -> new+new == new*2 }}
[22:13:04.122] Using cached result: d1f5a76e7a0b7c01 (SBV 9.2, Z3 4.8.7␣
```

(continues on next page)

(continued from previous page)

```
↪- 64 bit)
...
sawscript> prove_print (w4_unint_z3_using "qfnia" []) \
                                  {{ \(x:[8]) -> x+x == x*2 }}
[22:13:09.484] Caching result: 4ee451f8429c2dfe (What4 v1.3-29-g6c462cd␣
↪using qfnia, Z3 4.8.7 - 64 bit)
...
sawscript> print_solver_cache "d1f5a76e7a0b7c01"
[22:13:13.250] SHA:␣
↪d1f5a76e7a0b7c01bdfe7d0e1be82b4f233a805ae85a287d45933ed12a54d3eb
[22:13:13.250] - Result: unsat
[22:13:13.250] - Solver: "SBV->Z3"
[22:13:13.250] - Versions: SBV 9.2, Z3 4.8.7 - 64 bit
[22:13:13.250] - Last used: 2023-07-25 22:13:04.120351 UTC

sawscript> print_solver_cache "4ee451f8429c2dfe"
[22:13:16.727] SHA:␣
↪4ee451f8429c2dfefecb6216162bd33cf053f8e66a3b41833193529449ef5752
[22:13:16.727] - Result: unsat
[22:13:16.727] - Solver: "W4 ->z3"
[22:13:16.727] - Versions: What4 v1.3-29-g6c462cd using qfnia, Z3 4.8.7␣
↪- 64 bit
[22:13:16.727] - Last used: 2023-07-25 22:13:09.484464 UTC

sawscript> print_solver_cache_stats
[22:13:20.585] == Solver result cache statistics ==
[22:13:20.585] - 2 results cached in example.cache
[22:13:20.585] - 2 insertions into the cache so far this run (0 failed␣
↪attempts)
[22:13:20.585] - 1 usage of cached results so far this run (0 failed␣
↪attempts)
```

14.6 Other External Provers

In addition to the built-in automated provers already discussed, SAW supports more generic interfaces to other arbitrary theorem provers supporting specific interfaces.

- `external_aig_solver : String -> [String] -> ProofScript ()` supports theorem provers that can take input as a single-output AIGER file. The first argument is the name of the executable to run. The second argument is the list of command-line parameters to pass to that executable. Any element of this list equal to `"%f"` will be replaced with the name of the temporary AIGER file generated for the proof goal. The output from the solver is expected to be in DIMACS solution format.

- `external_cnf_solver : String -> [String] -> ProofScript ()` works similarly but for SAT solvers that take input in DIMACS CNF format and produce output in DIMACS solution format.

14.7 Offline Provers

For provers that must be invoked in more complex ways, or to defer proof until a later time, there are functions to write the current goal to a file in various formats, and then assume that the goal is valid through the rest of the script.

- `offline_aig : String -> ProofScript ()`
- `offline_cnf : String -> ProofScript ()`
- `offline_extcore : String -> ProofScript ()`
- `offline_smtlib2 : String -> ProofScript ()`
- `offline_unint_smtlib2 : [String] -> String -> ProofScript ()`

These support the AIGER, DIMACS CNF, shared SAWCore, and SMT-Lib v2 formats, respectively. The shared representation for SAWCore is described in the `saw-script` repository (https://github.com/GaloisInc/saw-script/blob/master/doc/extcore.md). The `offline_unint_smtlib2` command represents the folded subterms listed in its first argument as uninterpreted functions.

14.8 Finishing Proofs without External Solvers

Some proofs can be completed using unsound placeholders, or using techniques that do not require significant computation.

- `assume_unsat : ProofScript ()` indicates that the current goal should be assumed to be unsatisfiable. This is an alias for `assume_valid`. Users should prefer to use `admit` instead.
- `assume_valid : ProofScript ()` indicates that the current goal should be assumed to be valid. Users should prefer to use `admit` instead
- `admit : String -> ProofScript ()` indicates that the current goal should be assumed to be valid without proof. The given string should be used to record why the user has decided to assume this proof goal.
- `quickcheck : Int -> ProofScript ()` runs the goal on the given number of random inputs, and succeeds if the result of evaluation is always `True`. This is unsound, but can be helpful during proof development, or as a way to provide some evidence for the validity of a specification believed to be true but difficult or infeasible to prove.
- `trivial : ProofScript ()` states that the current goal should be trivially true. This tactic recognizes instances of equality that can be demonstrated by conversion alone. In particular it

is able to prove `EqTrue x` goals where `x` reduces to the constant value `True`. It fails if this is not the case.

14.9 Multiple Goals

The proof scripts shown so far all have a single implicit goal. As in many other interactive provers, however, SAWScript proofs can have multiple goals. The following commands can introduce or work with multiple goals. These are experimental and can be used only after `enable_experimental` has been called.

- `goal_apply : Theorem -> ProofScript ()` will apply a given introduction rule to the current goal. This will result in zero or more new subgoals.
- `goal_assume : ProofScript Theorem` will convert the first hypothesis in the current proof goal into a local `Theorem`
- `goal_insert : Theorem -> ProofScript ()` will insert a given `Theorem` as a new hypothesis in the current proof goal.
- `goal_intro : String -> ProofScript Term` will introduce a quantified variable in the current proof goal, returning the variable as a `Term`.
- `goal_when : String -> ProofScript () -> ProofScript ()` will run the given proof script only when the goal name contains the given string.
- `goal_exact : Term -> ProofScript ()` will attempt to use the given term as an exact proof for the current goal. This tactic will succeed whever the type of the given term exactly matches the current goal, and will fail otherwise.
- `split_goal : ProofScript ()` will split a goal of the form `Prelude.and prop1 prop2` into two separate goals `prop1` and `prop2`.

14.10 Proof Failure and Satisfying Assignments

The `prove_print` and `sat_print` commands print out their essential results (potentially returning a `Theorem` in the case of `prove_print`). In some cases, though, one may want to act programmatically on the result of a proof rather than displaying it.

The `prove` and `sat` commands allow this sort of programmatic analysis of proof results. To allow this, they use two types we haven't mentioned yet: `ProofResult` and `SatResult`. These are different from the other types in SAWScript because they encode the possibility of two outcomes. In the case of `ProofResult`, a statement may be valid or there may be a counter-example. In the case of `SatResult`, there may be a satisfying assignment or the statement may be unsatisfiable.

- `prove : ProofScript SatResult -> Term -> TopLevel ProofResult`
- `sat : ProofScript SatResult -> Term -> TopLevel SatResult`

To operate on these new types, SAWScript includes a pair of functions:

- `caseProofResult : {b} ProofResult -> b -> (Term -> b) -> b` takes a `ProofResult`, a value to return in the case that the statement is valid, and a function to run on the counter-example, if there is one.
- `caseSatResult : {b} SatResult -> b -> (Term -> b) -> b` has the same shape: it returns its first argument if the result represents an unsatisfiable statement, or its second argument applied to a satisfying assignment if it finds one.

14.11 AIG Values and Proofs

Most SAWScript programs operate on `Term` values, and in most cases this is the appropriate representation. It is possible, however, to represent the same function that a `Term` may represent using a different data structure: an And-Inverter-Graph (AIG). An AIG is a representation of a Boolean function as a circuit composed entirely of AND gates and inverters. Hardware synthesis and verification tools, including the ABC tool that SAW has built in, can do efficient verification and particularly equivalence checking on AIGs.

To take advantage of this capability, a handful of built-in commands can operate on AIGs.

- `bitblast : Term -> TopLevel AIG` represents a `Term` as an `AIG` by "blasting" all of its primitive operations (things like bit-vector addition) down to the level of individual bits.
- `load_aig : String -> TopLevel AIG` loads an `AIG` from an external AIGER file.
- `save_aig : String -> AIG -> TopLevel ()` saves an `AIG` to an external AIGER file.
- `save_aig_as_cnf : String -> AIG -> TopLevel ()` writes an `AIG` out in CNF format for input into a standard SAT solver.

CHAPTER 15

Extraction to the Rocq theorem prover

In addition to the (semi-)automatic and compositional proof modes already discussed above, SAW has support for exporting Cryptol and SAWCore values as terms to the Rocq proof assistant[5]. This is intended to support more manual proof efforts for properties that go beyond what SAW can support (for example, proofs requiring induction) or for connecting to preexisting formalizations in Rocq of useful algorithms (e.g. the fiat crypto library[6]).

This support consists of two related pieces. The first piece is a library of formalizations of the primitives underlying Cryptol and SAWCore and various supporting concepts that help bridge the conceptual gap between SAW and Rocq. The second piece is a term translation that maps the syntactic forms of SAWCore onto corresponding concepts in Rocq syntax, designed to dovetail with the concepts defined in the support library. SAWCore is a quite similar language to the core calculus underlying Rocq, so much of this translation is quite straightforward; however, the languages are not exactly equivalent, and there are some tricky cases that mostly arise from Cryptol code that can only be partially supported. We will note these restrictions later in the manual.

We currently support Rocq version 9.1.0.

15.1 Support Library

In order to make use of SAW's extraction capabilities, one must first compile the support library using Rocq so that the included definitions and theorems can be referenced by the extracted code. From the top directory of the SAW source tree, the source code for this support library can be found in the `saw-core-rocq/rocq` subdirectory. In this subdirectory you will find a `_RocqProject` and a

[5] https://rocq-prover.org
[6] https://github.com/mit-plv/fiat-crypto

`Makefile`. A simple `make` invocation should be enough to compile all the necessary files, assuming Rocq is installed and `rocq` is available in the user's `PATH`. HTML documentation for the support library can also be generated by `make html` from the same directory.

Once the library is compiled, the recommended way to import it into your subsequent development is by adding the following lines to your `_RocqProject` file:

```
-Q <SAWDIR>/saw-core-rocq/rocq/generated/CryptolToRocq CryptolToRocq
-Q <SAWDIR>/saw-core-rocq/rocq/handwritten/CryptolToRocq CryptolToRocq
```

Here `<SAWDIR>` refers to the location on your system where the SAWScript source tree is checked out. This will add the relevant library files to the `CryptolToRocq` namespace, where the extraction process will expect to find them.

The support library for extraction is broken into two parts: those files which are handwritten, versus those that are automatically generated. The handwritten files are generally fairly readable and are reasonable for human inspection; they define most of the interesting pipe-fitting that allows Cryptol and SAWCore definitions to connect to corresponding Rocq concepts. In particular the file `SAWCoreScaffolding.v` file defines most of the bindings of base types to Rocq types, and the `SAWCoreVectorsAsRocqVectors.v` defines the core bitvector operations. The automatically generated files are direct translations of the SAWCore source files (`saw-core/prelude/Prelude.sawcore` and `cryptol-saw-core/saw/Cryptol.sawcore`) that correspond to the standard libraries for SAWCore and Cryptol, respectively.

The autogenerated files are intended to be kept up-to-date with changes in the corresponding `sawcore` files, and end users should not need to generate them. Nonetheless, if they are out of sync for some reason, these files may be regenerated using the `saw` executable by running `(cd saw-core-rocq; saw saw/generate_scaffolding.saw)` from the top-level of the SAW source directory before compiling them with Rocq as described above.

You may also note some additional files and concepts in the standard library, such as `CompM.v`, and a variety of lemmas and definitions related to it. These definitions are related to the "heapster" system, which form a separate use-case for the SAWCore to Rocq translation. These definitions will not be used for code extracted from Cryptol.

15.2 Cryptol module extraction

There are several modes of use for the SAW to Rocq term extraction facility, but the easiest to use is whole Cryptol module extraction. This will extract all the definitions in the given Cryptol module, together with it's transitive dependencies, into a single Rocq module which can then be compiled and pulled into subsequence developments.

Suppose we have a Cryptol source file named `source.cry` and we want to generate a Rocq file named `output.v`. We can accomplish this by running the following commands in saw (either directly from the `saw` command prompt, or via a script file)

```
write_rocq_cryptol_module "source.cry" "output.v" [] [];
```

In this default mode, identifiers in the Cryptol source will be directly translated into identifiers in Rocq. This may occasionally cause problems if source identifiers clash with Rocq keywords or preexisting definitions. The third argument to `write_rocq_cryptol_module` can be used to remap such names if necessary by giving a list of `(in,out)` pairs of names. The fourth argument is a list of source identifiers to skip translating, if desired. Authoritative online documentation for this command can be obtained directly from the `saw` executable via `:help write_rocq_cryptol_module`.

The resulting "output.v" file will have some of the usual hallmarks of computer-generated code; it will have poor formatting and, explicit parenthesis and fully-qualified names. Thankfully, once loaded into Rocq, the Rocq pretty-printer will do a much better job of rendering these terms in a somewhat human-readable way.

15.3 Proofs involving uninterpreted functions

It is possible to write a Cryptol module that references uninterpreted functions by using the `primitive` keyword to declare them in your Cryptol source. Primitive Cryptol declarations will be translated into Rocq section variables; as usual in Rocq, uses of these section variables will be translated into additional parameters to the definitions from outside the section. In this way, consumers of the translated module can instantiate the declared Cryptol functions with corresponding terms in subsequent Rocq developments.

Although the Cryptol interpreter itself will not be able to compute with declared but undefined functions of this sort, they can be used both to provide specifications for functions to be verified with `llvm_verify` or `jvm_verify` and also for Rocq extraction.

For example, if I write the following Cryptol source file:

```
primitive f : Integer -> Integer

g : Integer -> Bool
g x = f (f x) > 0
```

After extraction, the generated term `g` will have Rocq type:

```
(Integer -> Integer) -> Integer -> Bool
```

15.4 Translation limitations and caveats

Translation from Cryptol to Rocq has a number of fundamental limitations that must be addressed. The most severe of these is that Cryptol is a fully general-recursive language, and may exhibit runtime errors directly via calls to the `error` primitive, or via partial operations (such as indexing a sequence

out-of-bounds). The internal language of Rocq, by contrast, is a strongly-normalizing language of total functions. As such, our translation is unable to extract all Cryptol programs.

15.4.1 Recursive programs

The most severe of the currently limitations for our system is that the translation is unable to translate any recursive Cryptol program. Doing this would require us to attempt to find some termination argument for the recursive function sufficient to satisfy Rocq; for now, no attempt is made to do so. if you attempt to extract a recursive function, SAW will produce an error about a "malformed term" with `Prelude.fix` as the head symbol.

Certain limited kinds of recursion are available via the `foldl` Cryptol primitive operation, which is translated directly into a fold operation in Rocq. This is sufficient for many basic iterative algorithms.

15.4.2 Type coercions

Another limitation of the translation system is that Cryptol uses SMT solvers during its typechecking process and uses the results of solver proofs to justify some of its typing judgments. When extracting these terms to Rocq, type equality coercions must be generated. Currently, we do not have a good way to transport the reasoning done inside Cryptol's typechecker into Rocq, so we just supply a very simple `Ltac` tactic to discharge these coercions (see `solveUnsafeAssert` in `CryptolPrimitivesForSAWCoreExtra.v`). This tactic is able to discover simple coercions, but for anything nontrivial it may fail. The result will be a typechecking failure when compiling the generated code in Rocq when the tactic fails. If you encounter this problem, it may be possible to enhance the `solveUnsafeAssert` tactic to cover your use case.

15.4.3 Error terms

A final caveat that is worth mentioning is that Cryptol can sometimes produce runtime errors. These can arise from explicit calls to the `error` primitive, or from partially defined operations (e.g., division by zero or sequence access out of bounds). Such instances are translated to occurrences of an unrealized Rocq axiom named `error`. In order to avoid introducing an inconsistent environment, the `error` axiom is restricted to apply only to inhabited types. All the types arising from Cryptol programs are inhabited, so this is no problem in principle. However, collecting and passing this information around on the Rocq side is a little tricky.

The main technical device we use here is the `Inhabited` type class; it simply asserts that a type has some distinguished inhabitant. We provide instances for the base types and type constructors arising from Cryptol, so the necessary instances ought to be automatically constructed when needed. However, polymorphic Cryptol functions add some complications, as type arguments must also come together with evidence that they are inhabited. The translation process takes care to add the necessary `Inhabited` arguments, so everything ought to work out. However, if Rocq typechecking of generated code fails with errors about `Inhabited` class instances, it likely represents some problem with this aspect of the translation.

CHAPTER 16

Formal Deprecation Process

SAW primitives, and thus their associated SAWScript built-ins, sometimes become obsolete or are found inadequate and replaced. The process by which that happens has three steps, as follows:

1. The decision is made to deprecate and eventually remove the objects in question. This can happen at the level of individual built-in elements (for example, when replacing a function with an awkward interface or unfortunate name) or at the level of internal units of functionality with possibly multiple built-ins affected. At this step the built-ins in question are marked for a deprecation warning. They remain available by default, but referring to them will trigger a warning.
2. The objects in question are made invisible by default. Now, referring to the affected built-ins will fail unless the `enable_deprecated` command is used. In that case referring to them will still produce a warning.
3. The objects in question are removed entirely and are no longer available.

In general any object or group of objects will move only one step per release; that is, something first marked deprecated (so it warns) in saw-script 1.2 will not disappear by default before saw-script 1.3 and not be removed entirely before saw-script 1.4. The time frame may be longer depending on the needs of downstream users, the complexity of migration, and the cost/impact of keeping the deprecated code in the system.

We may move faster if circumstances dictate, but hope not to need to.

Objects that have never appeared in a release, or that have never moved past experimental may be removed without first being deprecated. However, we aim to avoid this in cases where the objects in question have gotten substantial use despite their formal status.

CHAPTER 17

Appendices

17.1 Glossary

> **Warning**
>
> This section is under construction!

17.2 Command Reference

> **Warning**
>
> This section is under construction!

17.3 REPL Reference

The primary mechanism for interacting with SAW is through the `saw` executable included as part of the standard binary distribution. With no arguments, `saw` starts a read-evaluate-print loop (REPL) that allows the user to interactively evaluate commands in the SAWScript language. With one file name argument, it executes the specified file as a SAWScript program.

In addition to a file name, the `saw` executable accepts several command-line options:

`-h, -?, --help`
: Print a help message.

-V, --version
Show the version of the SAWScript interpreter.

-c path, --classpath=path
Specify a colon-delimited list of paths to search for Java classes.

-i path, --import-path=path
Specify a colon-delimited list of paths to search for imports. Note that paths can also be specified using the `SAW_IMPORT_PATH` environment variable.

-t, --extra-type-checking
Perform extra type checking of intermediate values.

-I, --interactive
Run interactively (with a REPL). This is the default if no other arguments are specified.

-B file, --batch=file
Start the REPL, but load the commands from the given file instead of standard input. This allows automated use of the REPL `:`-commands and other REPL-specific affordances in scripts.

-j path, --jars=path
Specify a colon-delimited list of paths to `.jar` files to search for Java classes.

-b path, --java-bin-dirs
Specify a colon-delimited list of paths to search for a Java executable.

-d num, --sim-verbose=num
Set the verbosity level of the Java and LLVM simulators.

-v num, --verbose=num
Set the verbosity level of the SAWScript interpreter.

--clean-mismatched-versions-solver-cache[=path]
Run the `clean_mismatched_versions_solver_cache` command on the solver cache at the given path, or if no path is given, the solver cache at the value of the `SAW_SOLVER_CACHE_PATH` environment variable, then exit. See the section **Caching Solver Results** for a description of the `clean_mismatched_versions_solver_cache` command and the solver caching feature in general.

SAW also uses several environment variables for configuration:

CRYPTOLPATH
Specify a colon-delimited list of directory paths to search for Cryptol imports (including the Cryptol prelude).

PATH
If the `--java-bin-dirs` option is not set, then the `PATH` will be searched to find a Java executable.

SAW_IMPORT_PATH
Specify a colon-delimited list of directory paths to search for imports. Note that paths can also be specified using the `-i`/`--import-path` command-line options.

SAW_JDK_JAR
Specify the path of the `.jar` file containing the core Java libraries. Note that that is not necessary if the `--java-bin-dirs` option or the `PATH` environment variable is used, as SAW can use this information to determine the location of the core Java libraries' `.jar` file.

SAW_SOLVER_CACHE_PATH
Specify a path at which to keep a cache of solver results obtained during calls to certain tactics. A cache is not created at this path until it is needed. See the section **Caching Solver Results** for more detail about this feature.

On Windows, semicolon-delimited lists are used instead of colon-delimited lists.

17.3.1 Using `:search`

The REPL `:search` command takes one or more type patterns as arguments, and searches the current value namespace (including functions and builtins) for objects that mention types matching all the patterns given. In practice this is mostly useful for searching for builtins.

Type patterns are type names extended with _ as a wildcard. Thus for example `[_]` matches any list type. You may also forall-bind type variables before the patterns using the `{a}` syntax. The scope of such bindings is the whole search.

Complex patterns should be written in parentheses; otherwise they become syntactically ambiguous. Note in particular that parentheses are required to treat a type constructor application to another type as a single search term. Without parentheses, `:search` would treat the unapplied type constructor and the other type as two separate search terms. For instance, `:search ProofScript Int` will search for objects mentioning both `ProofScript` (applied to anything) and `Int`. To search for objects that mention the type `ProofScript Int`, write `:search (ProofScript Int)`.

Type variables in the search patterns are matched as follows:

- Already-bound type variables (typedef names, certain builtin types) must match exactly.
- Free type variables match any type, but all occurrences must match the same type. Thus for example `[a] -> a` matches `sum : [Int] -> Int` and `head : {t} [t] -> t` but not `length : {t} [t] -> Int`.

  This is true across all patterns in the same search; searching for `[a]`, `[b]`, and `a -> b` will match `map : {a, b} (a -> b) -> [a] -> [b]`, as well as `pam : {a, b} [a] -> (a -> b) -> [b]` if you define such a thing. But it won't match `mapFirst : {a, b, c} (a -> b) [(a, c)] -> [(b, c)]`.

  Perhaps unfortunately, it *will* match `intNth : [Int] -> Int -> Int`. The search logic does not require distinct patterns to match distinct parts of the target type, nor is there a way to prevent it from picking the same substitution for both `a` and `b`. (Neither of these behaviors is entirely desirable and might be improved in the future.)
- Forall-bound type variables in the pattern are matched in the same way as free type variables, but will *only* match forall-bound type variables in the search space. Thus, `:search {a} (a`

`-> a)` will match `id : {a} a -> a` but not `inc: Int -> Int`, while `:search (a -> a)` will match both. This is helpful to search specifically for polymorphic functions.

Because SAWScript functions are curried, searching for `Int -> Int` will match `String -> Int -> Int`. However, it will not match `Int -> String -> Int`. The best way to search for a function that takes `Int` in *any* argument position and also returns `Int` is by searching for both `Int -> _` and `_ -> Int`: `:search (Int -> _) (_ -> Int)`.

There is, however, no good way yet to search for a function that takes two `Int`s in arbitrary argument positions. Searching for `Int -> Int -> _` will only find functions where the two arguments are adjacent, `Int -> _ -> Int -> _` will only find functions where one other argument is between them, and searching for `Int -> _` twice falls afoul of the limitation where two patterns can match the same thing.

17.4 Deprecated Items

Warning

This section is under construction!

- `addsimp' : Term -> Simpset -> Simpset`
- `addsimps' : [Term] -> Simpset -> Simpset`

  These functions unconditionally added rules represented as `Term` values to a `Simpset`. When used in a proof, these functions make the correctness of the added rules a side condition of the proof process's soundness.

17.5 SAWScript Language Reference

Warning

This section is under construction!